IBM **Quantum**

Qiskit Global Summer School 2026

Lecture 1: introduction to quantum computing

Roland de Putter - Quantum Algorithm Engineering, IBM

Outline

1. What is quantum computing?
2. Quantum speedups
3. Quantum applications
4. The quantum utility era
5. Fault-tolerant quantum computing

What is quantum computing?

Quantum computing is a fundamentally new technology that uses the laws of quantum mechanics to unlock previously unsolvable problems

IBM **Quantum**

Bits and classical logic circuits

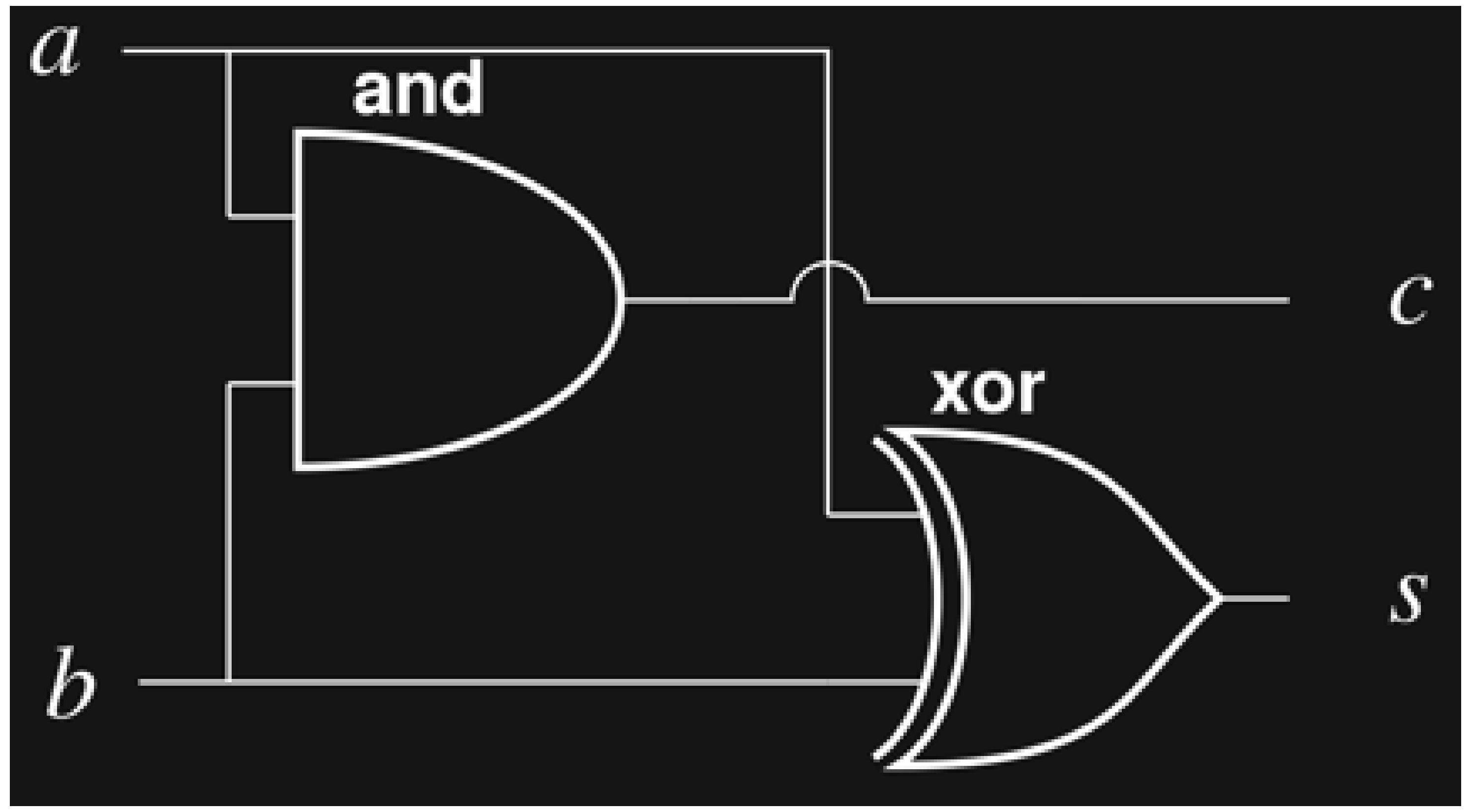
0

•

•

1

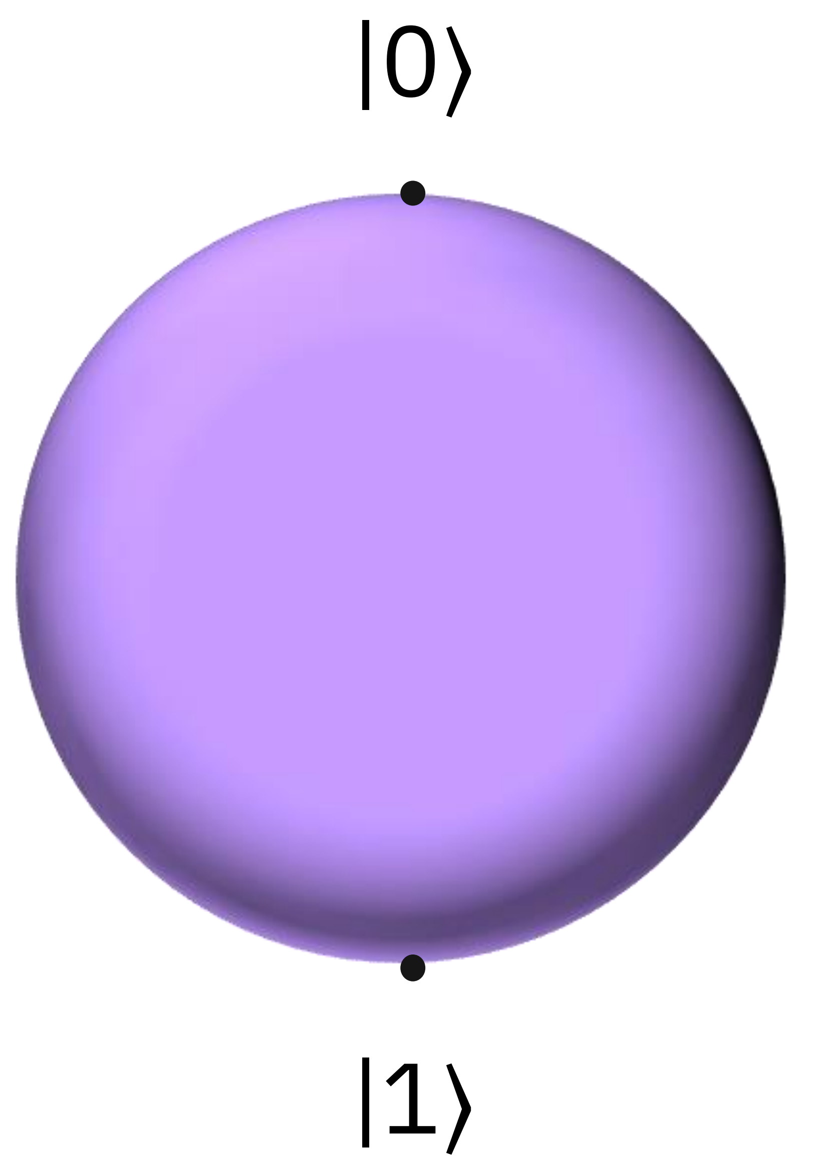
A bit is a controllable classical object that is the unit of information



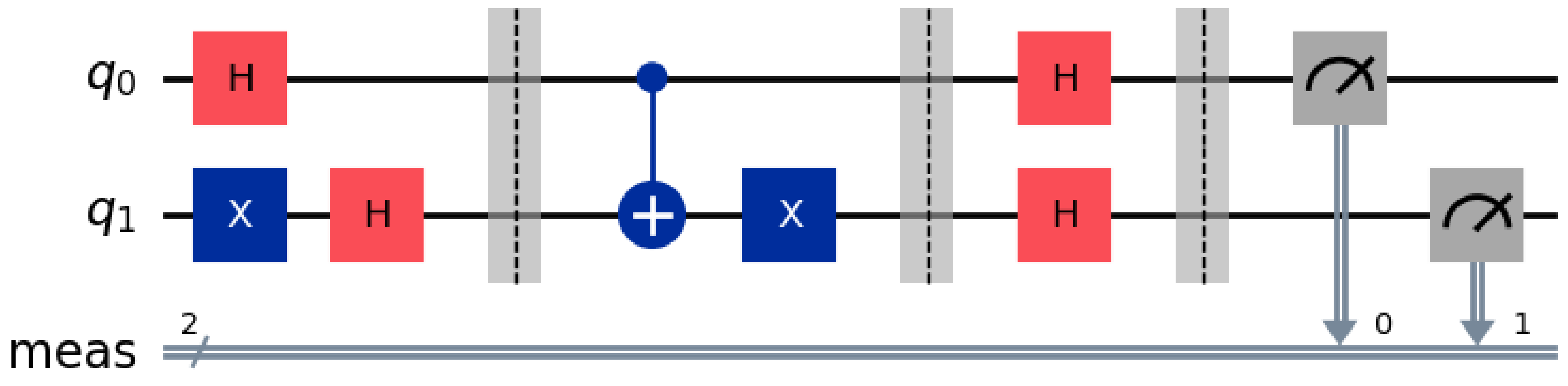
A classical logic circuit is a set of gate operations on bits and is the unit of computation

Quantum bits (qubits) and quantum circuits

IBM **Quantum**

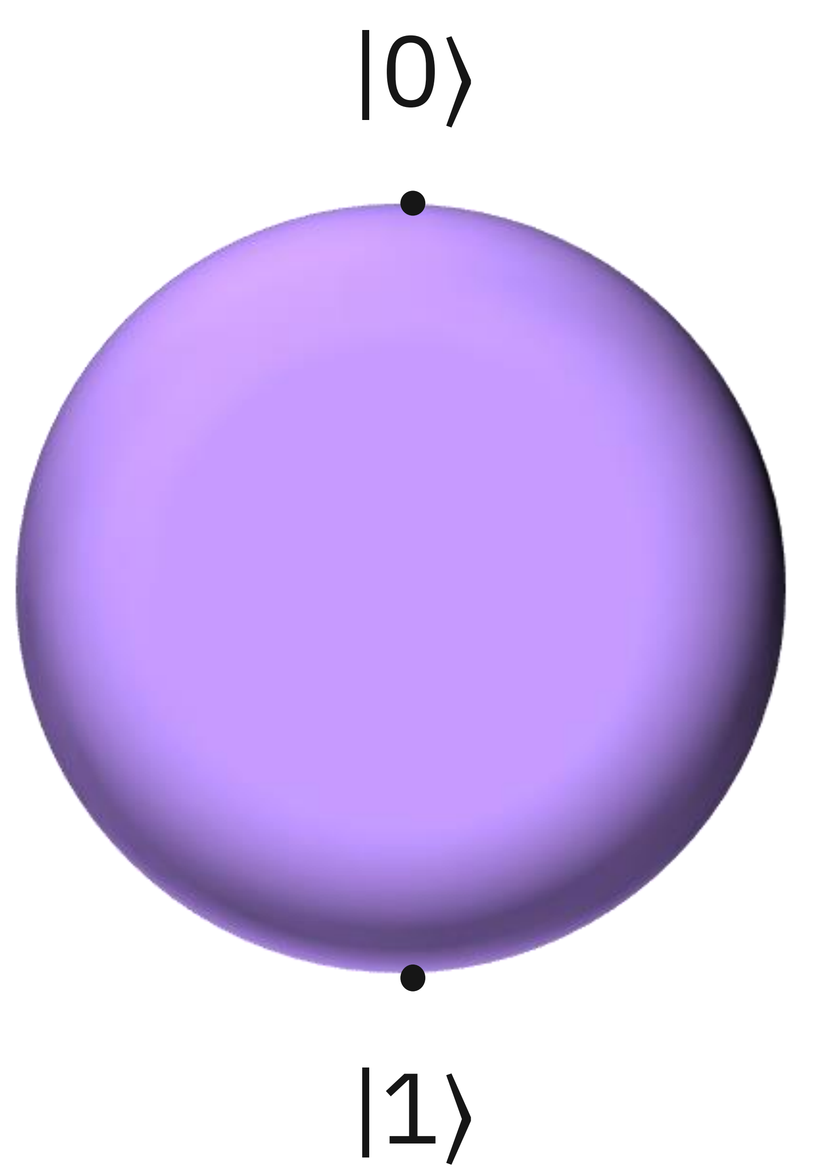


A quantum bit or qubit is a controllable quantum object that is the unit of information



A quantum circuit is a set of quantum gate operations on qubits and is the unit of computation

Qubits can be in quantum *superposition*



A qubit in superposition is a quantum state that is a combination of $|0\rangle$ and $|1\rangle$

$$a|0\rangle + b|1\rangle$$

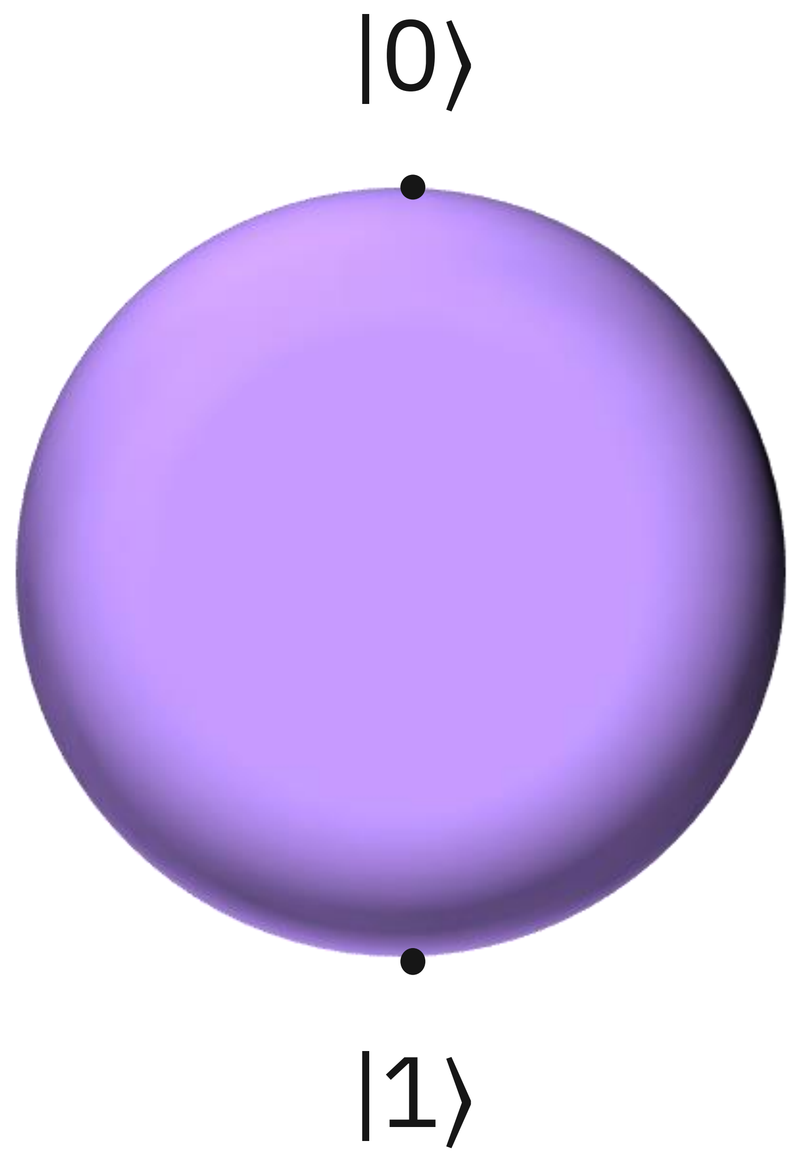
where a and b are complex numbers and

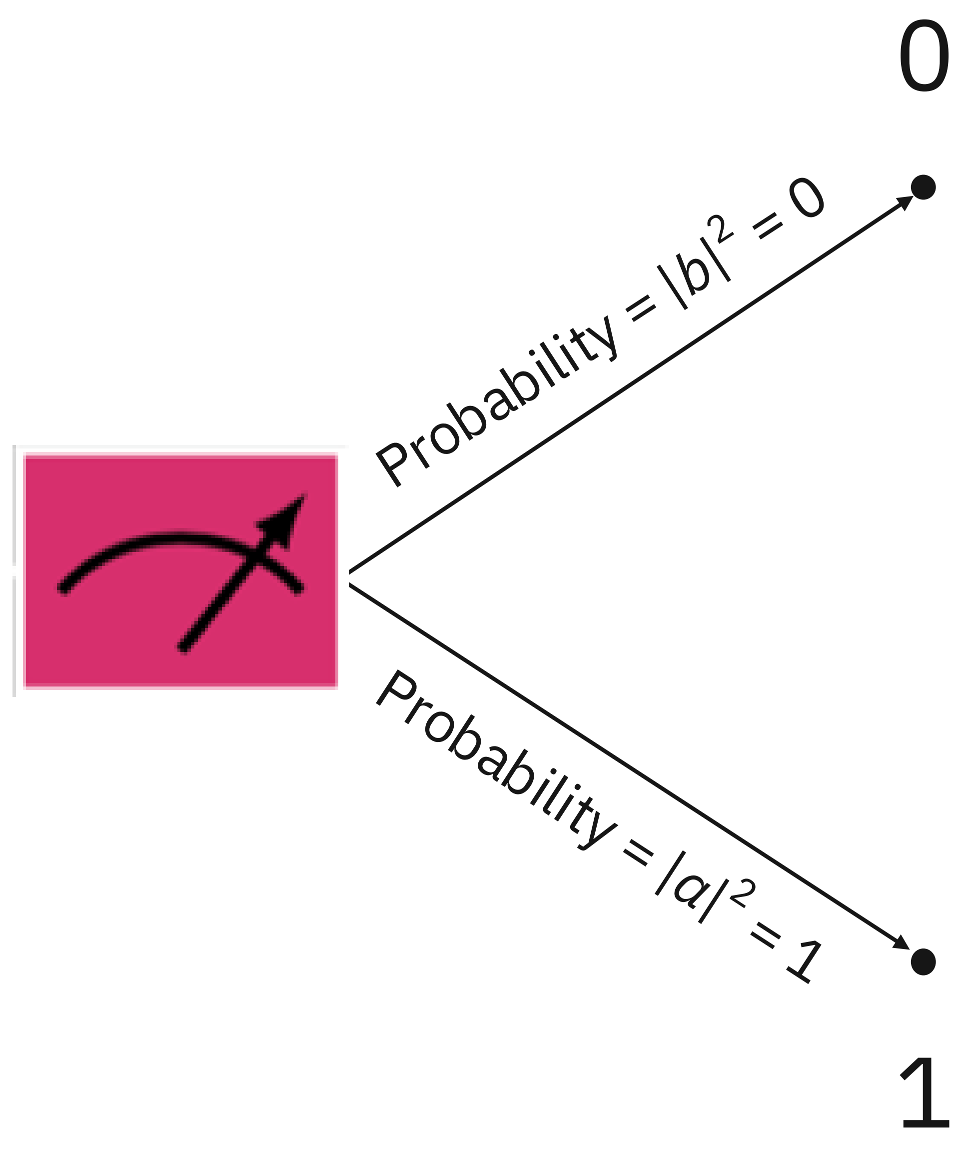
$$|a|^2 + |b|^2 = 1$$

This is what people mean when they say that a qubit can be "0 and 1 at the same time."

Qubit measurement/readout

IBM **Quantum**





A qubit's state is in a superposition of $|0\rangle$ and $|1\rangle$:

$$a|0\rangle + b|1\rangle$$

When we measure a qubit, it becomes ***0* OR *1*** based on probability.

Qubit measurement/readout
Born rule

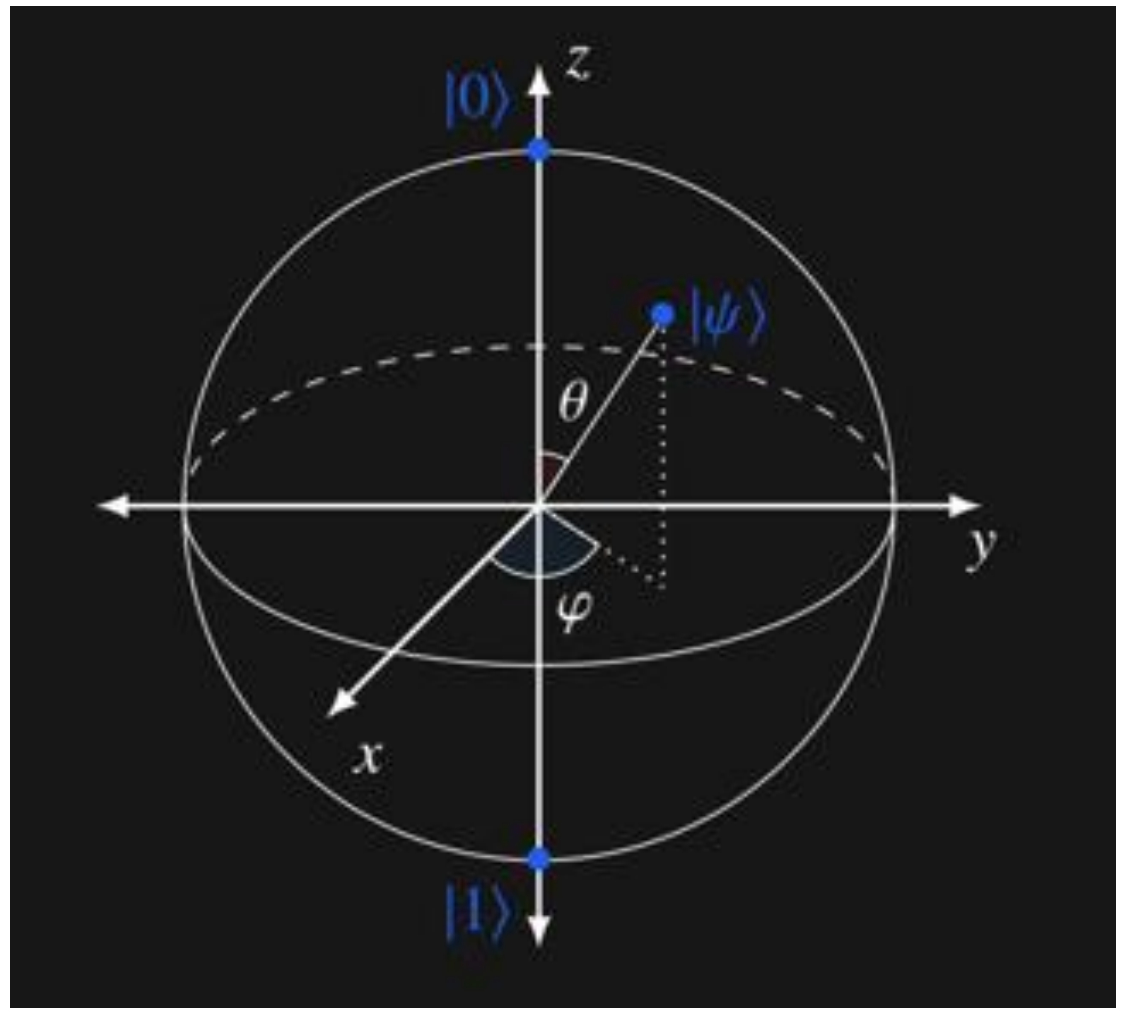
Measurement is forcing the qubit's state

$$a|0\rangle + b|1\rangle$$

to $|0\rangle$ or $|1\rangle$ by observing it, where

$|a|^2$ is the probability we will get $|0\rangle$ when we measure

$|b|^2$ is the probability we will get $|1\rangle$ when we measure



For example,

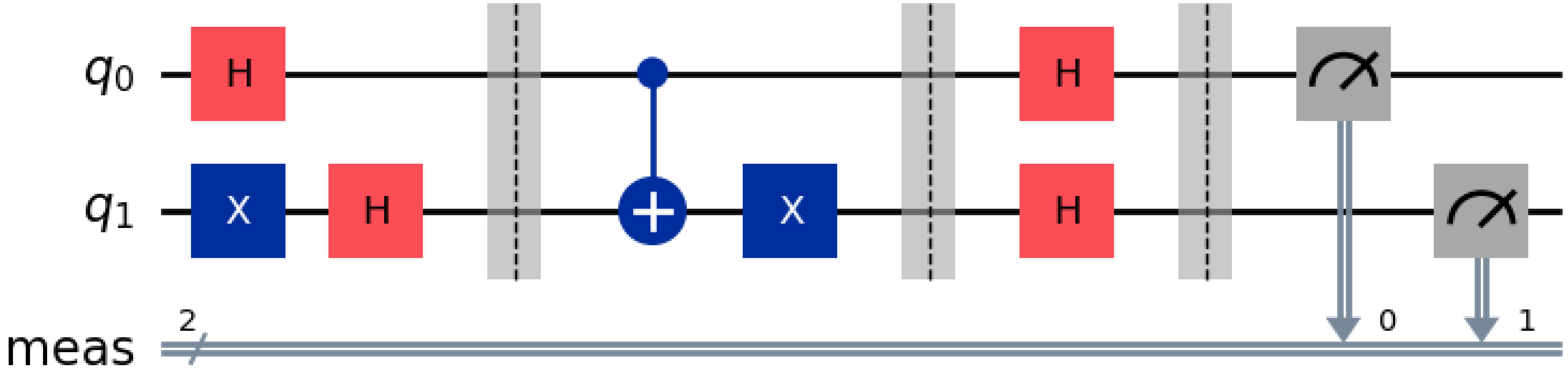
$$\frac{\sqrt{3}}{2}|0\rangle - \frac{1}{2}i\,|1\rangle$$

has a 75% chance of becoming $|0\rangle$.

Quantum computation workflow

IBM **Quantum**

- Start with all qubits in zero-state $|00 \ldots 0\rangle$
- Execute circuit to create final quantum state of qubits
- Measure qubits → get bitstrings
- Use measured bitstrings to compute quantity of interest (bitstring(s), expectation values, …)



Quantum parallelism and the dimensionality of Hilbert space

1 qubit	$a_0\|0\rangle + a_1\|1\rangle$	superposition of 2 states
2 qubits	$a_{00}\|00\rangle + \mathrm{a}_{01}\|01\rangle + a_{10}\|10\rangle + a_{11}\|11\rangle$	superposition of 4 states
n qubits	$\sum_{x=0}^{2^n-1} a_x\,\|x\rangle = a_0\|0\rangle + a_1\|1\rangle + \cdots + a_{2^n-1}\|2^n-1\rangle$	superposition of 2^n states

The number of states that can be realized in a superposition grows *exponentially* with the number of qubits

Equivalently: *a quantum state of n qubits stores 2^n complex amplitudes, whereas n classical bits are described by only n numbers (each 0 or 1) so the space of states accessed by a quantum computer is exponentially larger*

Modern laptop

30 qubits

IBM Summit
Supercomputer

48 qubits

All classical computers
on earth connected

60 qubits

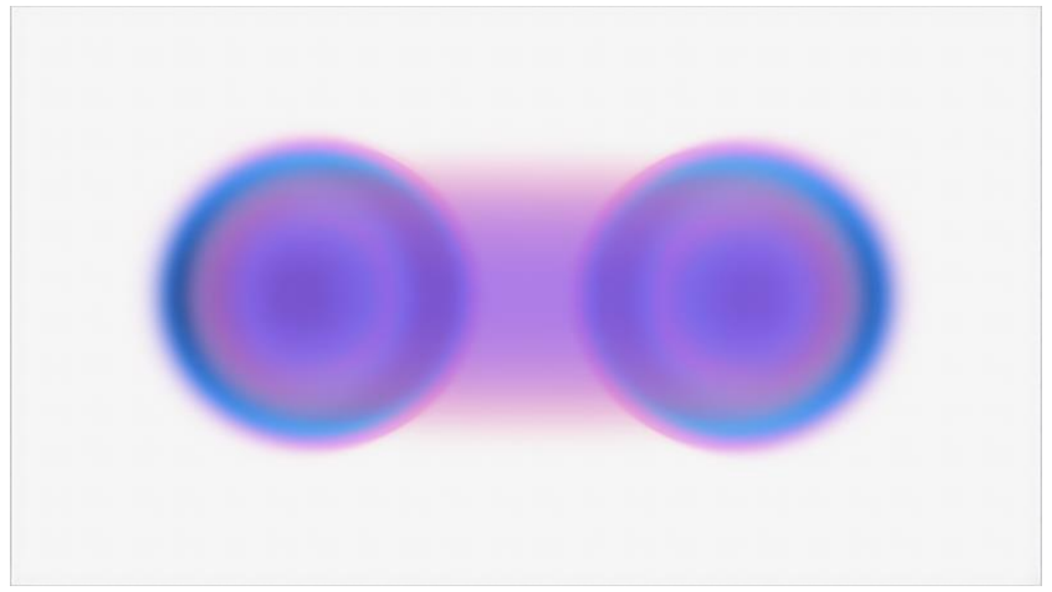
2029 IBM Quantum Starling

200* qubits

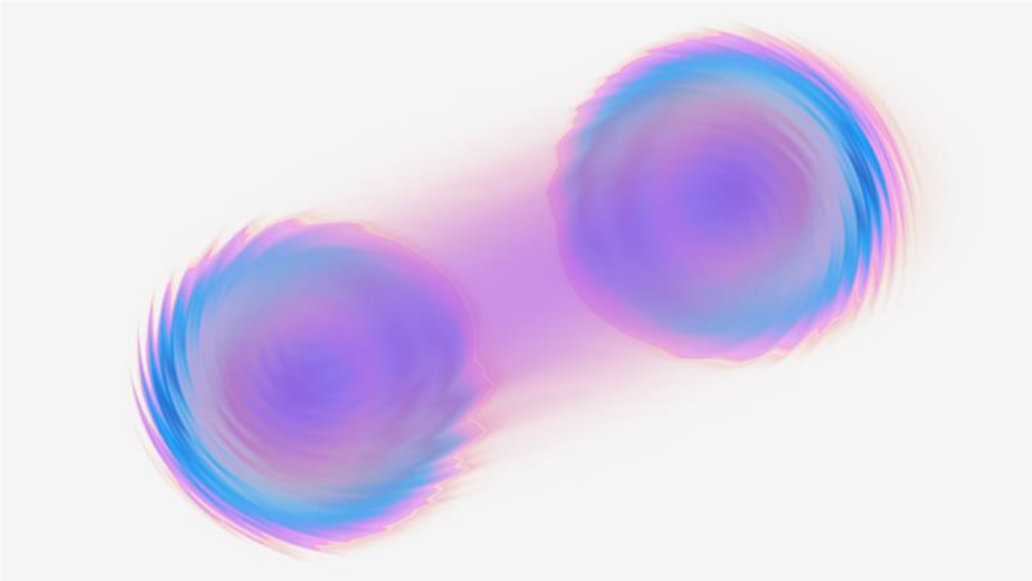
* Logical qubits

Caution: to achieve actual quantum speedups, one needs to make clever use of...

Superposition
A quantum system existing in a complex linear combination of two states until it is measured



Entanglement
Information shared jointly between entangled pairs or groups



Interference
The combining of amplitudes so that possibilities can reinforce or cancel one another

Quantum algorithms typically exploit hidden global structure of the problem, e.g. symmetry or global correlations

Quantum speedups

Query gates

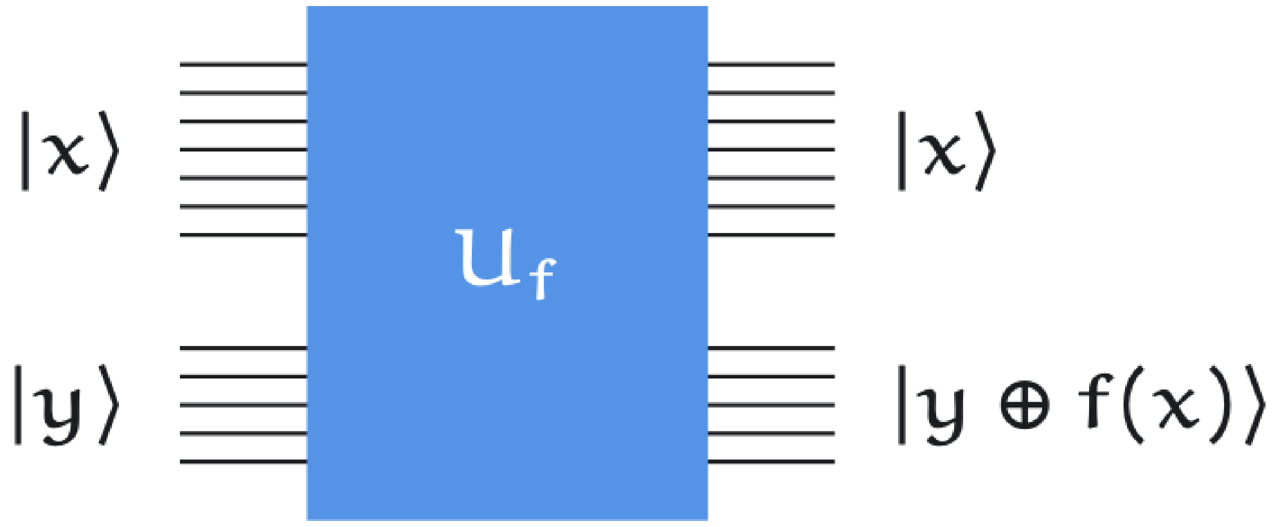
Definition

The ***query gate*** U_f for any function $f : \Sigma^n \rightarrow \Sigma^m$ is defined as

$$U_f(|y\rangle|x\rangle) = |y \oplus f(x)\rangle|x\rangle$$

for all $x \in \Sigma^n$ and $y \in \Sigma^m$.

In circuit diagrammatic form U_f operates like this:



Query gates

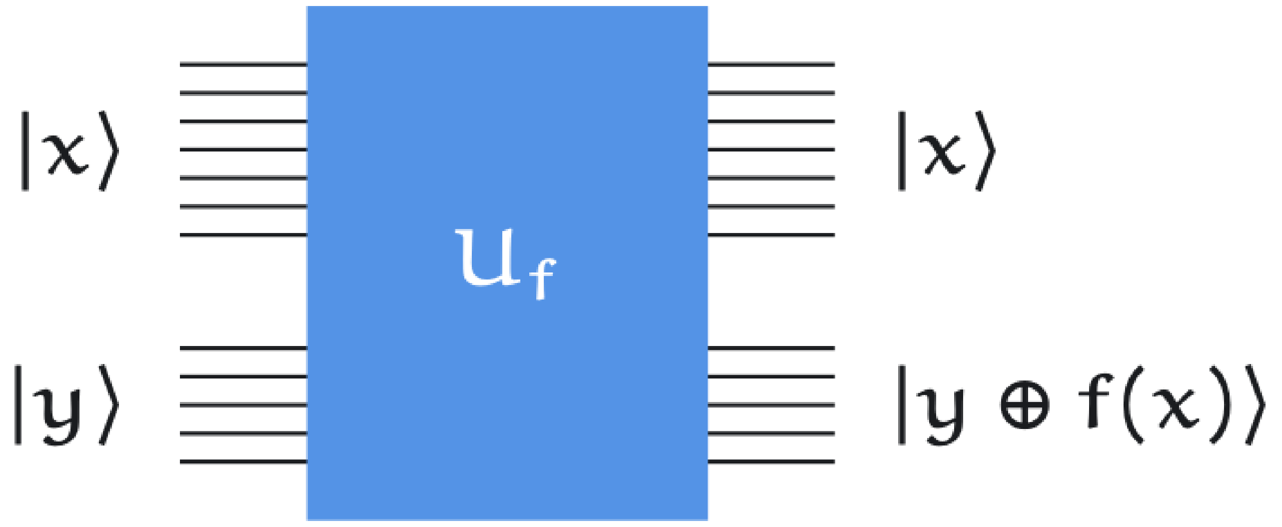
Definition

The *query gate* U_f for any function $f : \Sigma^n \rightarrow \Sigma^m$ is defined as

$$U_f(|y\rangle|x\rangle) = |y \oplus f(x)\rangle|x\rangle$$

for all $x \in \Sigma^n$ and $y \in \Sigma^m$.

In circuit diagrammatic form U_f operates like this:



In particular,

$$U_f|0\rangle|x\rangle = |f(x)\rangle|x\rangle$$

What quantum computing does *not* do

It is a common misconception that quantum computers can take any *calculation and achieve an exponential speedup by running exponentially many instances of the calculation in parallel*

Example of what does not work: naïve parallelization

1 - A function $f(x)$ can be evaluated by a quantum circuit: $|0\rangle|x\rangle \rightarrow |f(x)\rangle|x\rangle$

2 – Concretely, for all inputs x:

$$|0\rangle|0\rangle \rightarrow |f(0)\rangle|0\rangle$$
$$|0\rangle|1\rangle \rightarrow |f(1)\rangle|1\rangle$$
$$|0\rangle|2\rangle \rightarrow |f(2)\rangle|2\rangle$$
$$\vdots$$
$$|0\rangle|2^n-1\rangle \rightarrow |f(2^n-1)\rangle|2^n-1\rangle$$

3 – By linearity, if the input is an equal superposition of all possible inputs x, the output is a superposition of all possible input-output pairs:

$$|0\rangle\frac{1}{\sqrt{2^n}}\sum_{x=0}^{2^n-1}|x\rangle \rightarrow \frac{1}{\sqrt{2^n}}\sum_{x=0}^{2^n-1}|f(x)\rangle\,|x\rangle$$

4 - Can we therefore evaluate exponentially many function input-output pairs with only a single circuit execution?

Example of what does not work: naïve parallelization

1 - A function $f(x)$ can be evaluated by a quantum circuit: $|0\rangle|x\rangle \rightarrow |f(x)\rangle|x\rangle$

2 – Concretely, for all inputs x:

$$|0\rangle|0\rangle \rightarrow |f(0)\rangle|0\rangle$$
$$|0\rangle|1\rangle \rightarrow |f(1)\rangle|1\rangle$$
$$|0\rangle|2\rangle \rightarrow |f(2)\rangle|2\rangle$$
$$\vdots$$
$$|0\rangle|2^{n-1}\rangle \rightarrow |f(2^{n-1})\rangle|2^{n-1}\rangle$$

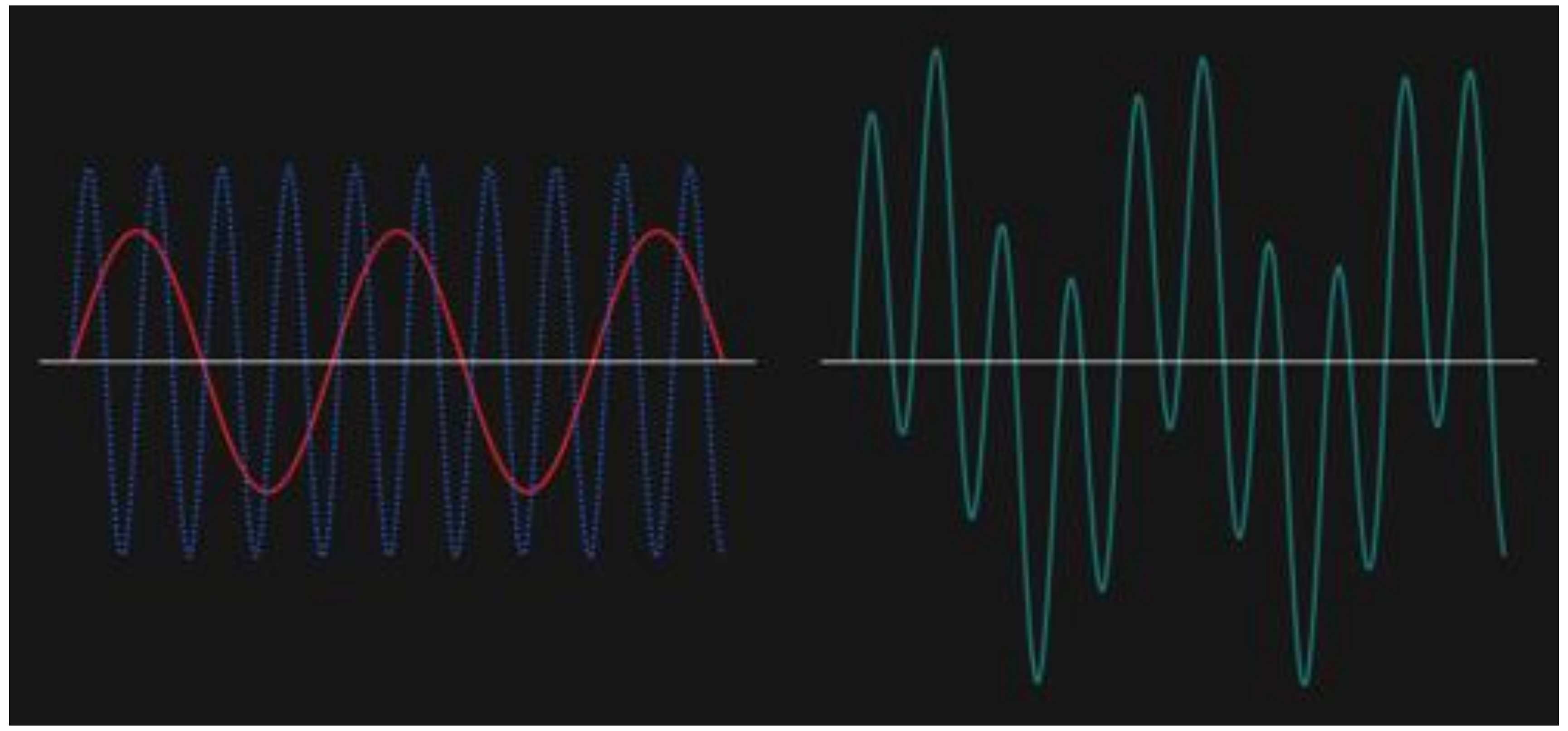
3 – By linearity, if the input is an equal superposition of all possible inputs x, the output is a superposition of all possible input-output pairs:

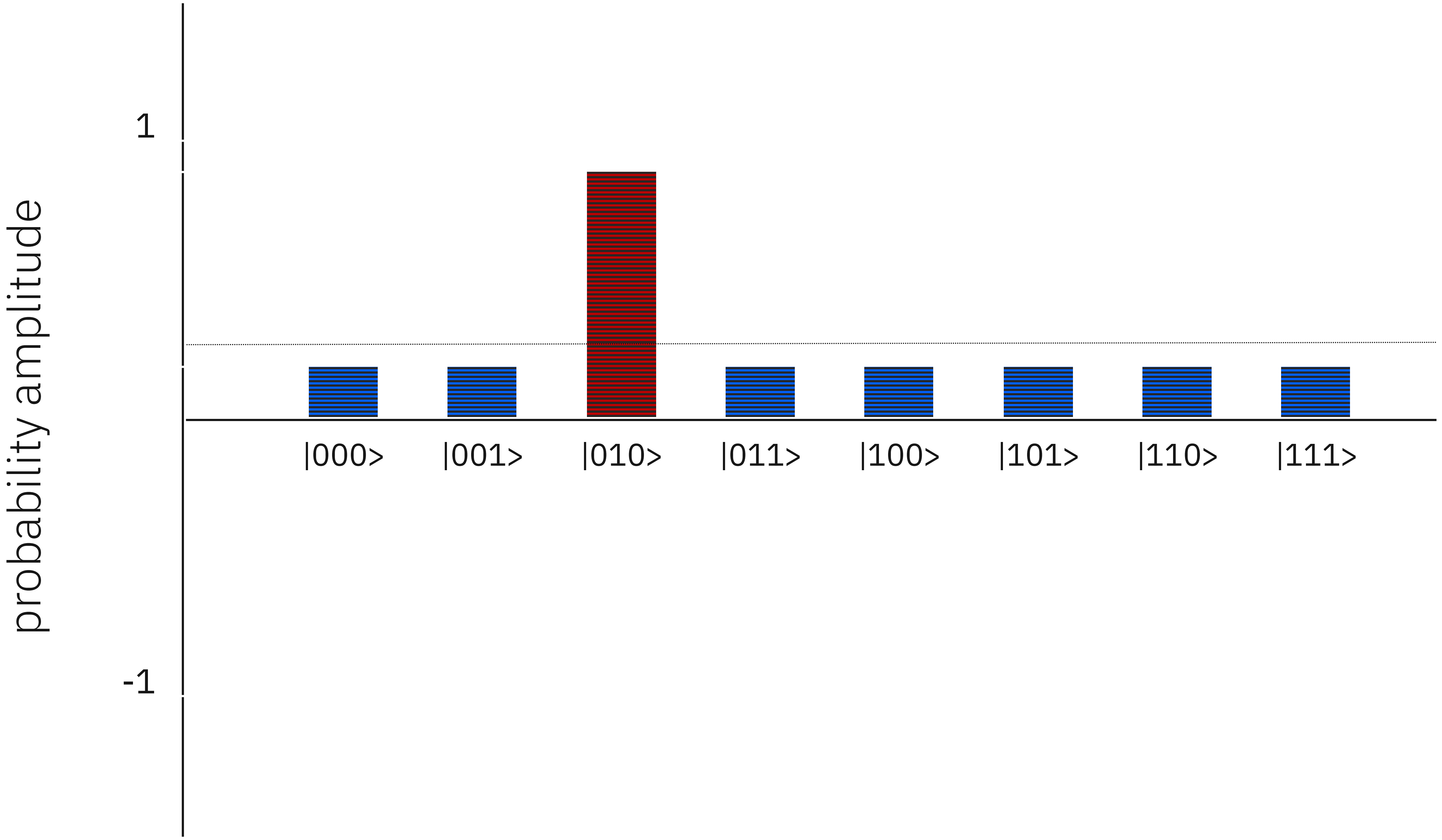
$$|0\rangle\frac{1}{\sqrt{2^n}}\sum_{x=0}^{2^n-1}|x\rangle \rightarrow \frac{1}{\sqrt{2^n}}\sum_{x=0}^{2^n-1}|f(x)\rangle\,|x\rangle$$

4 - Can we therefore evaluate exponentially many function input-output pairs with only a single circuit execution? No. Measurement randomly "collapses" the wave function to only a single input-output pair.

$$\frac{1}{\sqrt{2^n}}\sum_{x=0}^{2^n-1}|f(x)\rangle\,|x\rangle \rightarrow |f(a)\rangle\,|a\rangle$$, for a single, random input a

Good quantum algorithms use *interference* to increase the probability of getting the right answer and decrease the chance of getting the wrong one





Example of what *does* work

Deutsch's algorithm:

using interference to achieve the first real quantum speedup

Deutsch's problem

Deutsch's problem is very simple — it's the Parity problem for functions of the form $f : \Sigma \rightarrow \Sigma$.

First example of a quantum speedup

There are four functions of the form $f : \Sigma \rightarrow \Sigma$:

a	$f_1(a)$
0	0
1	0

a	$f_2(a)$
0	0
1	1

a	$f_3(a)$
0	1
1	0

a	$f_4(a)$
0	1
1	1

The functions f_1 and f_4 are *constant* while f_2 and f_3 are *balanced.*

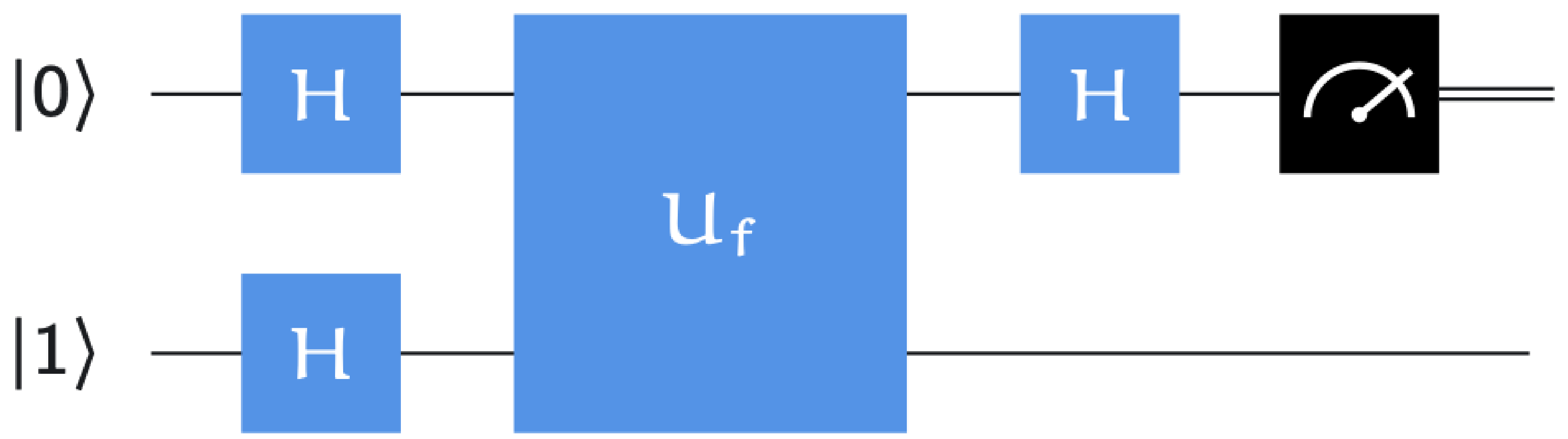
Deutsch's problem

Input: $f : \Sigma \rightarrow \Sigma$

Output: 0 if f is constant, 1 if f is balanced

Deutsch’s algorithm

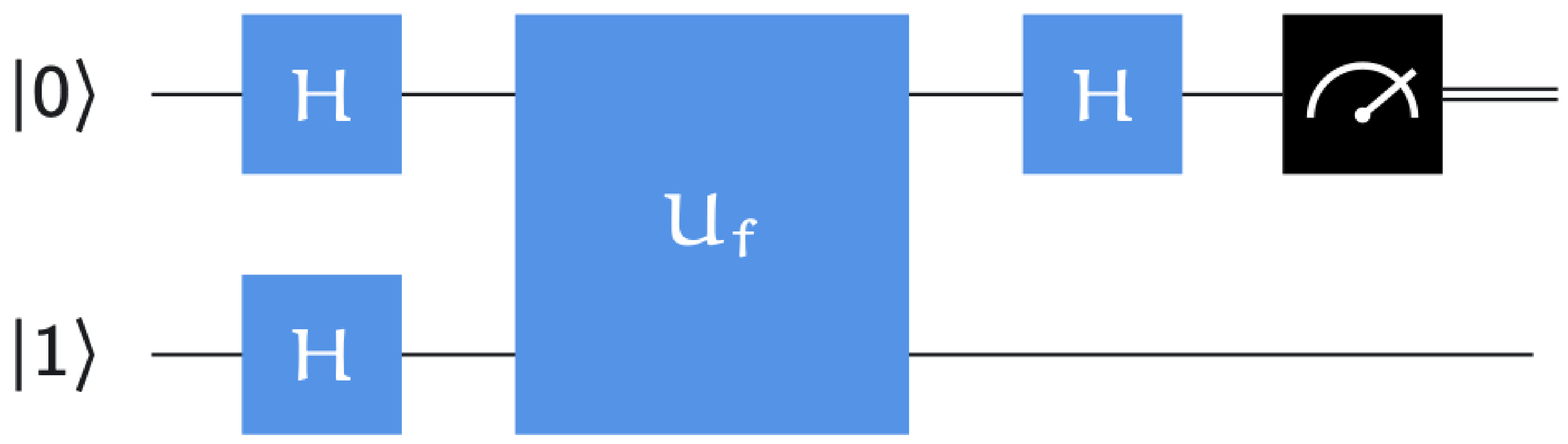
Deutsch’s algorithm solves Deutsch’s problem using a single query.



$$\begin{cases} 0 & \text{if } f \text{ is constant} \\ 1 & \text{if } f \text{ is balanced} \end{cases}$$

Deutsch's algorithm

Deutsch's algorithm solves Deutsch's problem using a single query.



$$\begin{cases} 0 & \text{if } f \text{ is constant} \\ 1 & \text{if } f \text{ is balanced} \end{cases}$$

Preliminaries

Effect of Hadamard gate H:

$$H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \equiv |+\rangle$$

$$H|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) \equiv |-\rangle$$

Query gate:

$$U_f|y\rangle|x\rangle = |y \oplus f(x)\rangle|x\rangle$$

Phase kickback:

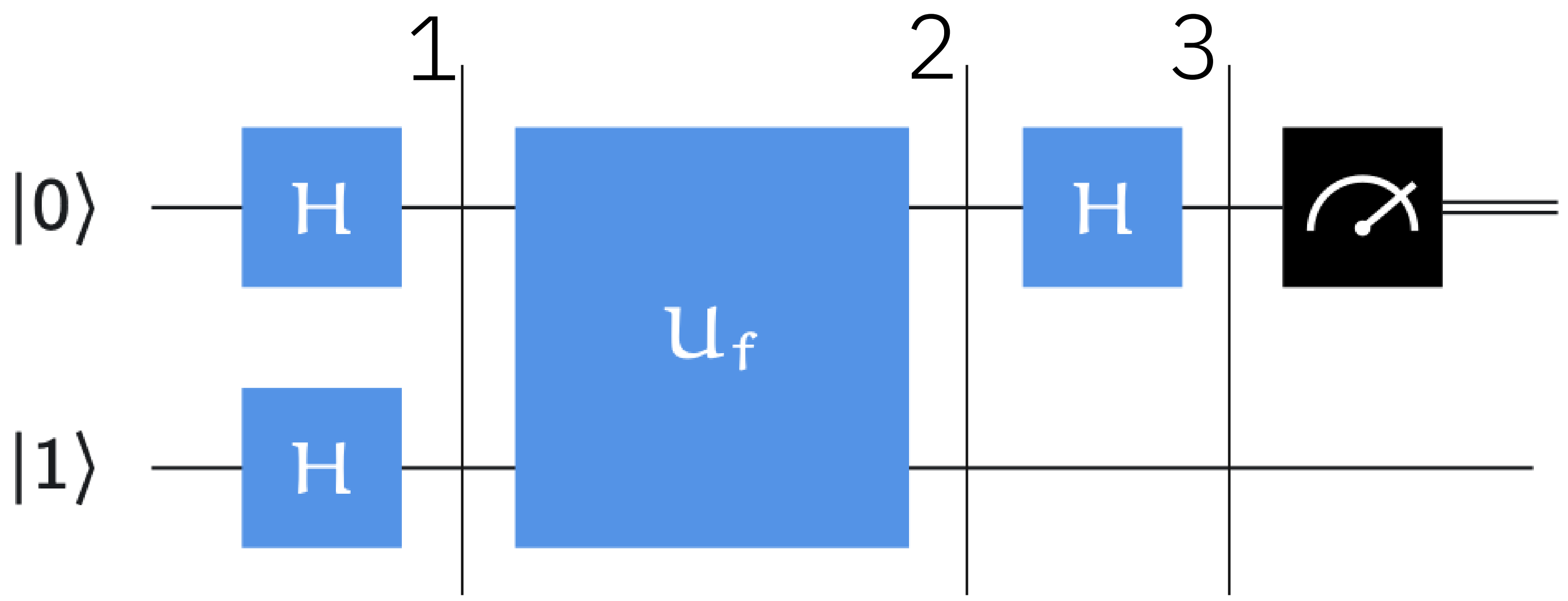
$$U_f|-\rangle|x\rangle = (-1)^{f(x)}|-\rangle|x\rangle$$

So $U_f|-\rangle|x\rangle = |-\rangle|x\rangle$ if $f(x) = 0$

and $U_f|-\rangle|x\rangle = -|-\rangle|x\rangle$ if $f(x) = 1$

Deutsch's algorithm

Deutsch's algorithm solves Deutsch's problem using a single query.



$$\begin{cases} 0 & \text{if f is constant} \\ 1 & \text{if f is balanced} \end{cases}$$

$$|\pi_1\rangle = |-\rangle \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$

Preliminaries

Effect of Hadamard gate H:

$$H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \equiv |+\rangle$$
$$H|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) \equiv |-\rangle$$

Query gate:

$$U_f|\mathrm{y}\rangle|x\rangle = |y \oplus f(x)\rangle|x\rangle$$

Phase kickback:

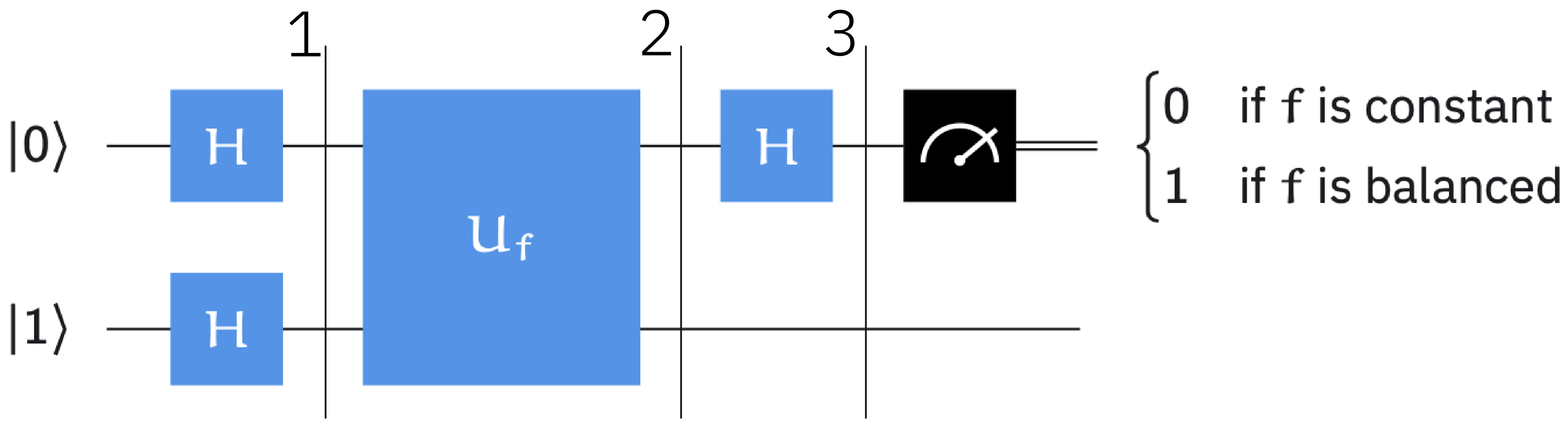
$$U_f|-\rangle|x\rangle = (-1)^{f(x)}|-\rangle|x\rangle$$

So $U_f|-\rangle|x\rangle = |-\rangle|x\rangle$ if $f(x) = 0$

and $U_f|-\rangle|x\rangle = -|-\rangle|x\rangle$ if $f(x) = 1$

Deutsch's algorithm

Deutsch's algorithm solves Deutsch's problem using a single query.



$$|\pi_1\rangle = |-\rangle \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$

$$|\pi_2\rangle = |-\rangle\frac{1}{\sqrt{2}}\left((-1)^{f(0)}|0\rangle + (-1)^{f(1)}|1\rangle\right)$$

Preliminaries

Effect of Hadamard gate H:

$$H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \equiv |+\rangle$$

$$H|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) \equiv |-\rangle$$

Query gate:

$$U_f|y\rangle|x\rangle = |y \oplus f(x)\rangle|x\rangle$$

Phase kickback:

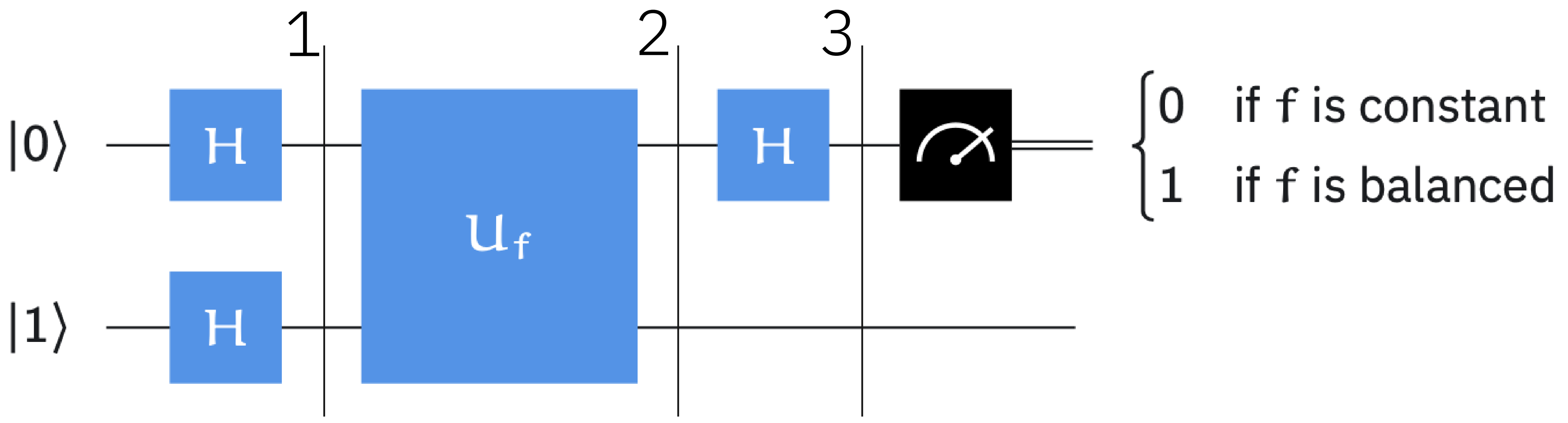
$$U_f|-\rangle|x\rangle = (-1)^{f(x)}|-\rangle|x\rangle$$

So $U_f|-\rangle|x\rangle = |-\rangle|x\rangle$ if $f(x) = 0$

and $U_f|-\rangle|x\rangle = -|-\rangle|x\rangle$ if $f(x) = 1$

Deutsch's algorithm

Deutsch's algorithm solves Deutsch's problem using a single query.



$$|\pi_1\rangle = |-\rangle \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$

$$|\pi_2\rangle = |-\rangle \frac{1}{\sqrt{2}}\left((-1)^{f(0)}|0\rangle + (-1)^{f(1)}|1\rangle\right)$$

$$|\pi_3\rangle = |-\rangle \frac{1}{\sqrt{2}}\left[(-1)^{f(0)}\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) + (-1)^{f(1)}\frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)\right]$$

Preliminaries

Effect of Hadamard gate H:

$$H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \equiv |+\rangle$$

$$H|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) \equiv |-\rangle$$

Query gate:

$$U_f|y\rangle|x\rangle = |y \oplus f(x)\rangle|x\rangle$$

Phase kickback:

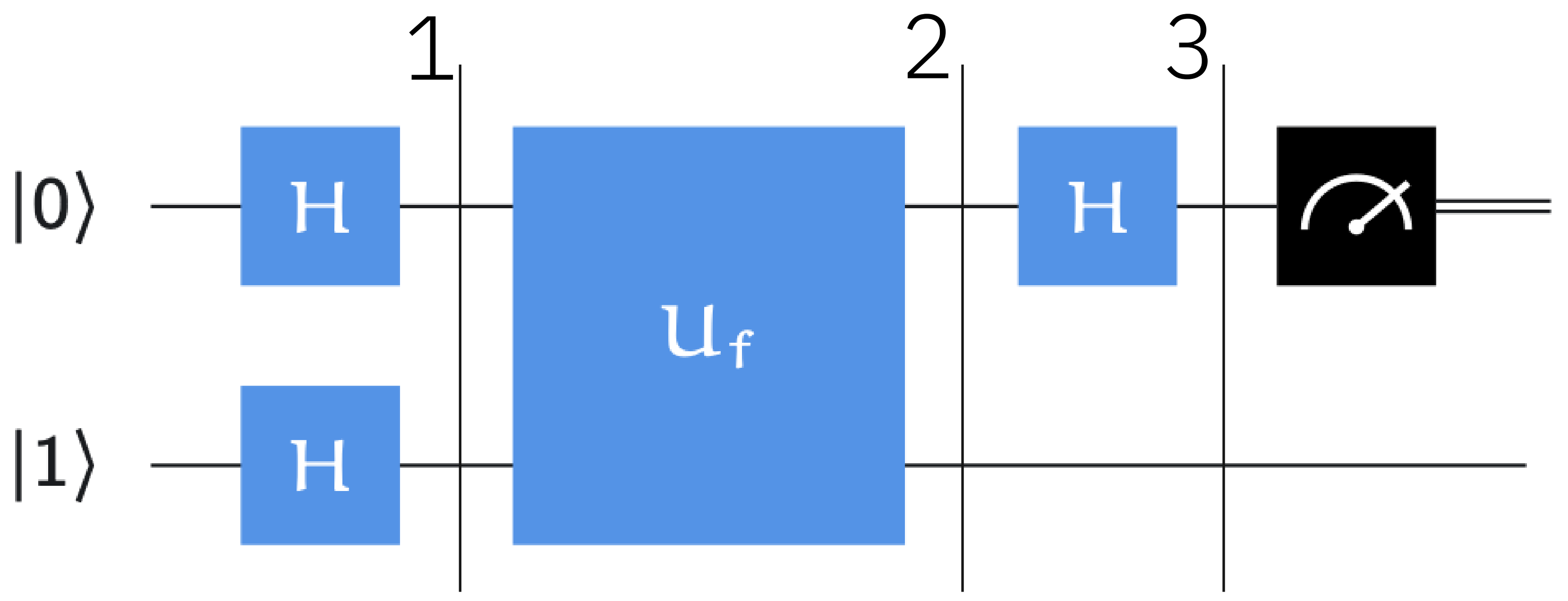
$$U_f|-\rangle|x\rangle = (-1)^{f(x)}|-\rangle|x\rangle$$

So $U_f|-\rangle|x\rangle = |-\rangle|x\rangle$ if $f(x) = 0$

and $U_f|-\rangle|x\rangle = -|-\rangle|x\rangle$ if $f(x) = 1$

Deutsch's algorithm

Deutsch's algorithm solves Deutsch's problem using a single query.



$$\begin{cases} 0 & \text{if } f \text{ is constant} \\ 1 & \text{if } f \text{ is balanced} \end{cases}$$

$$|\pi_1\rangle = |-\rangle \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$

$$|\pi_2\rangle = |-\rangle\frac{1}{\sqrt{2}}\left((-1)^{f(0)}|0\rangle + (-1)^{f(1)}|1\rangle\right)$$

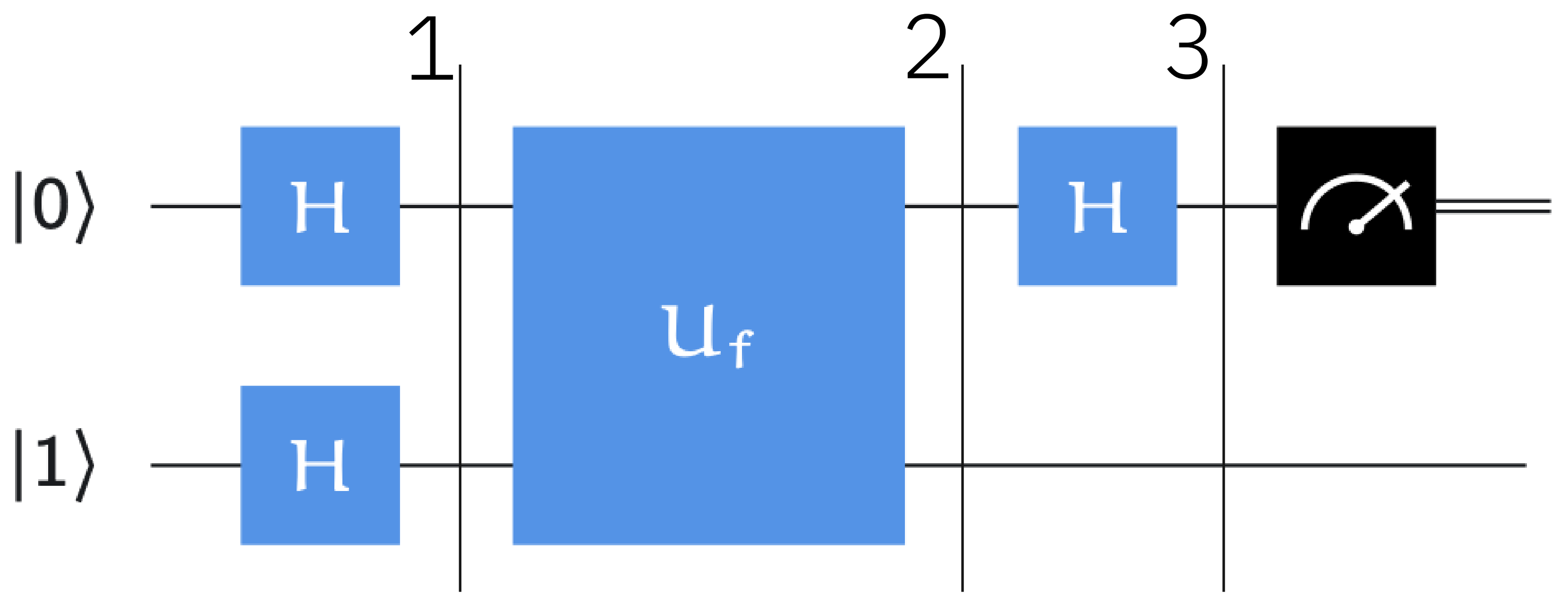
$$|\pi_3\rangle = |-\rangle\frac{1}{\sqrt{2}}[(-1)^{f(0)}\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) + (-1)^{f(1)}\frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)]$$

$$= |-\rangle\left[\tfrac{1}{2}\left((-1)^{f(0)} + (-1)^{f(1)}\right)|0\rangle + \left[\tfrac{1}{2}\left((-1)^{f(0)} - (-1)^{f(1)}\right)|1\rangle\right]$$

Interference

Deutsch’s algorithm

Deutsch’s algorithm solves Deutsch’s problem using a single query.



$$\begin{cases} 0 & \text{if } f \text{ is constant} \\ 1 & \text{if } f \text{ is balanced} \end{cases}$$

$$|\pi_1\rangle = |-\rangle \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$

$$|\pi_2\rangle = |-\rangle \frac{1}{\sqrt{2}}\left((-1)^{f(0)}|0\rangle + (-1)^{f(1)}|1\rangle\right)$$

$$|\pi_3\rangle = |-\rangle \frac{1}{\sqrt{2}}\left[(-1)^{f(0)}\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) + (-1)^{f(1)}\frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)\right]$$

$$= |-\rangle\left[\tfrac{1}{2}\left((-1)^{f(0)} + (-1)^{f(1)}\right)|0\rangle + \left[\tfrac{1}{2}\left((-1)^{f(0)} - (-1)^{f(1)}\right)|1\rangle\right]$$

Interference

$$= (-1)^{f(0)}|0\rangle|f(0) \oplus f(1)\rangle$$

Computational speedups characterized in terms of asymptotic scaling of computation time

IBM **Quantum**

In theory, anything that can be computed on a quantum computer, can also be computed on a classical computer (Church-Turing)

In practice, some problems that can be solved on quantum computers are impossible to crack with known classical algorithms

What matters are the resources needed to do a computation, i.e. can the calculation be performed efficiently?

The resources are often quantified in terms of **asymptotic scaling** of computation time/number of operations with problem size

Examples

- Polynomial time: $O(nc)$
- Exponential time: $O(c^n)$

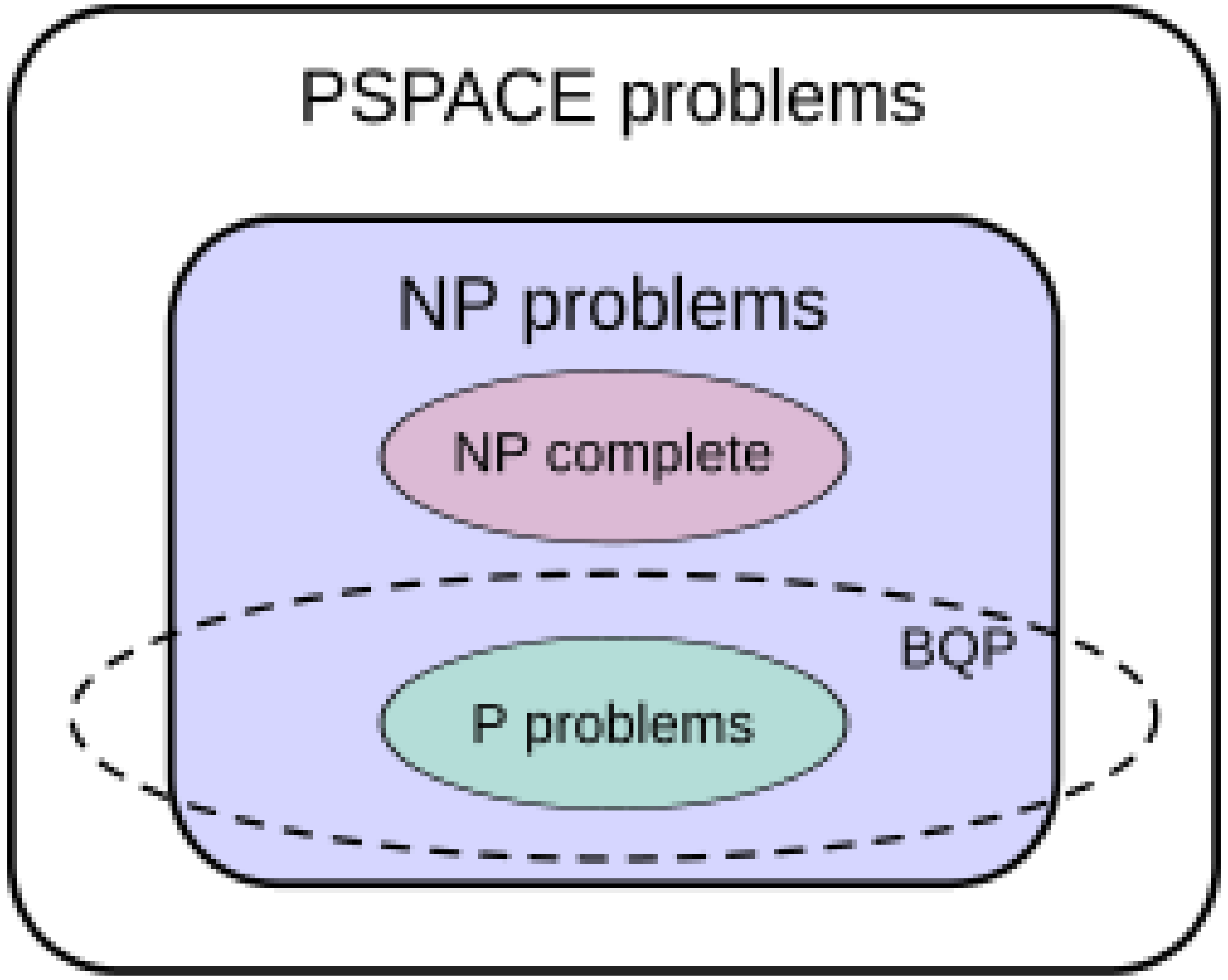
where n is the problem size

Computational complexity classes – quick overview

P: polynomial time
Can be solved classically in polynomial time

NP: non-deterministic polynomial time
Can be verified classically in polynomial time

BQP: bounded-error **quantum** polynomial time
Can be solved with high probability in polynomial time on a quantum computer



The Deutsch-Jozsa problem

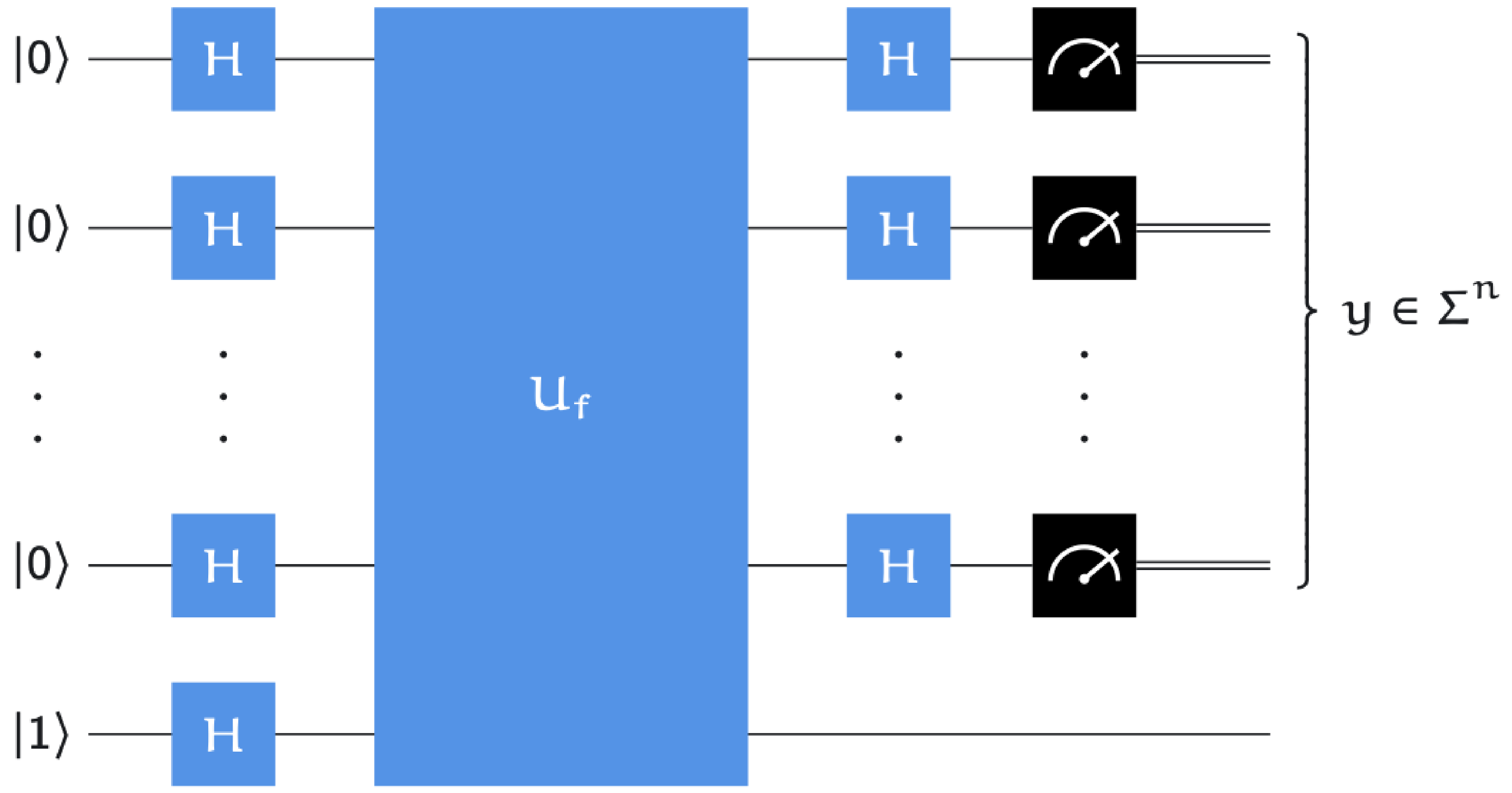
Deutsch-Jozsa problem

Input: $f : \Sigma^n \rightarrow \Sigma$

Promise: f is either constant or balanced

Output: 0 if f is constant, 1 if f is balanced

First example of an **exponential** quantum speedup (for deterministic algorithms)



Output: 0 if $y = 0^n$ and 1 otherwise.

Early algorithm milestones established that quantum computing leads to major speedups w.r.t. classical algorithms

Deutsch (1985)

Deutsch-Jozsa (1992)

Simon’s algorithm (1994)

Shor’s algorithm (1994)

Quantum phase estimation (1995)

Grover’s search (1996)

Quantum Algorithm Zoo

By Stephen Jordan,
with 435 contributions

Speedup* is measured in terms of asymptotic scaling

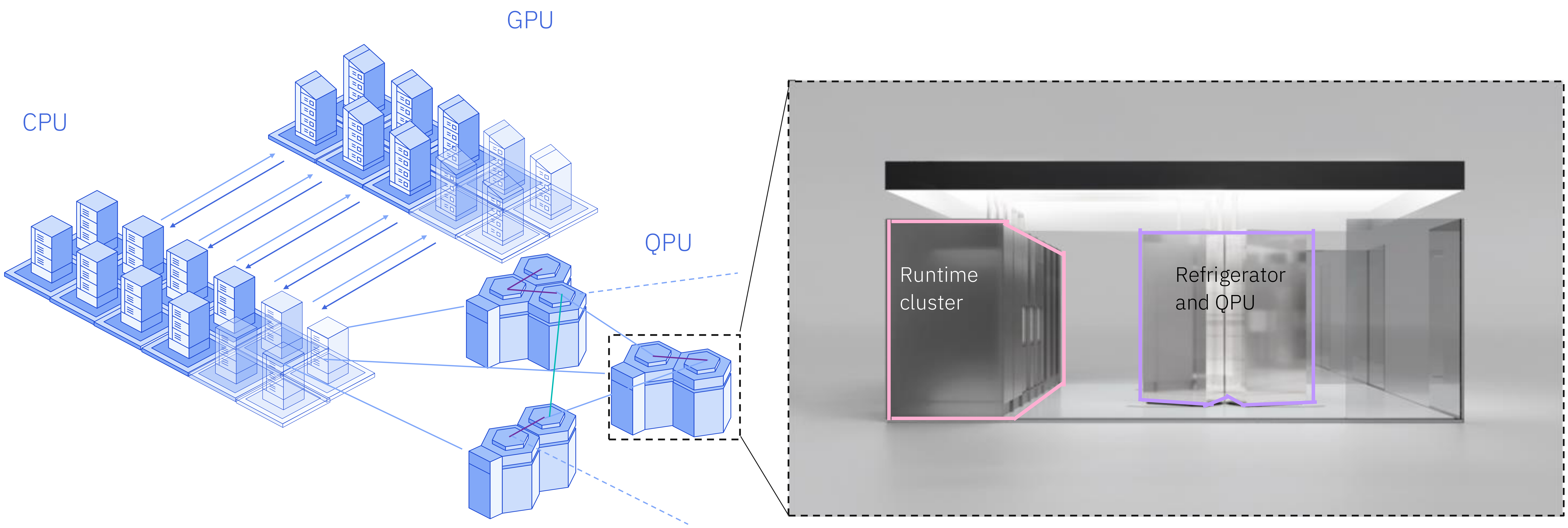
* Many subtleties (relative to best classical, in query complexity, …)

Algorithm: Factoring Speedup: Superpolynomial	Algorithm: Quantum Simulation Speedup: Superpolynomial	Algorithm: Searching Speedup: Polynomial
Algorithm: Abelian Hidden Subgroup Speedup: Superpolynomial	Algorithm: Machine learning Speedup: Varies	Algorithm: Verifying Matrix Products Speedup: Polynomial
Algorithm: Pattern matching Speedup: Superpolynomial	Algorithm: String Rewriting Speedup: Superpolynomial	Algorithm: Knot Invariants Speedup: Superpolynomial

quantumalgorithmzoo.org

Quantum-centric supercomputing: combining quantum with classical

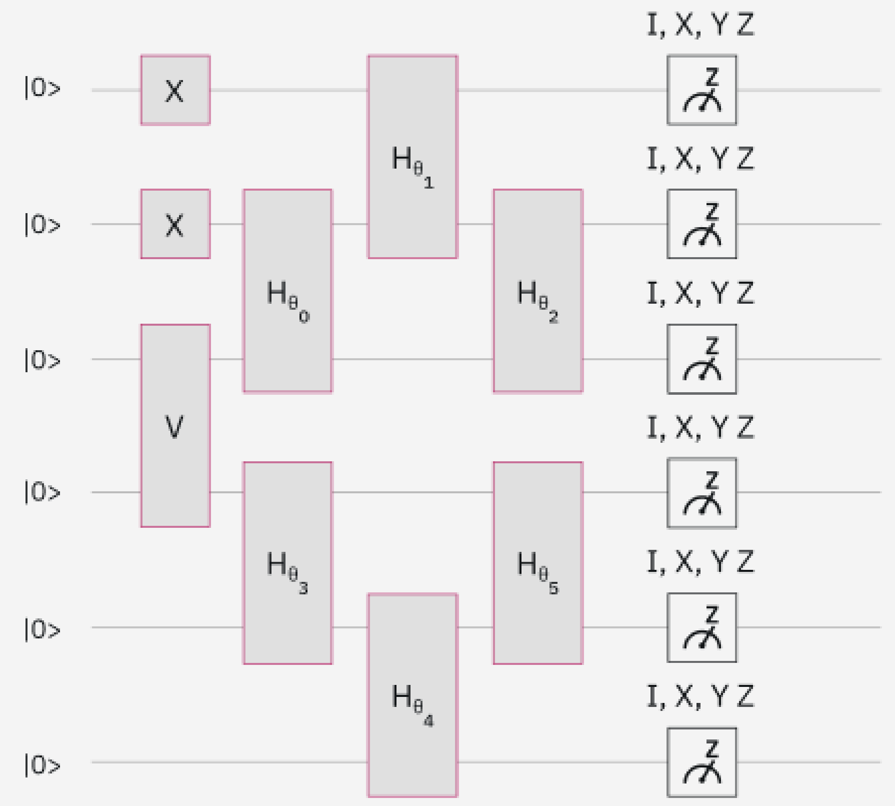
Delivering impactful quantum computing requires the interplay of quantum and classical resources at scale



Quantum applications

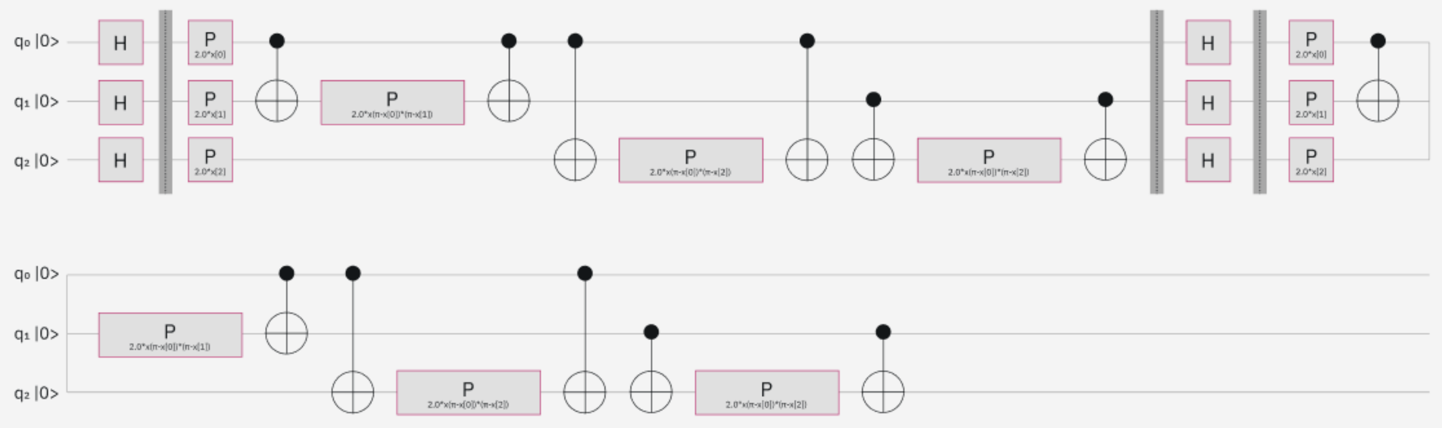
Broad problem areas where quantum computing adds value

Simulating nature



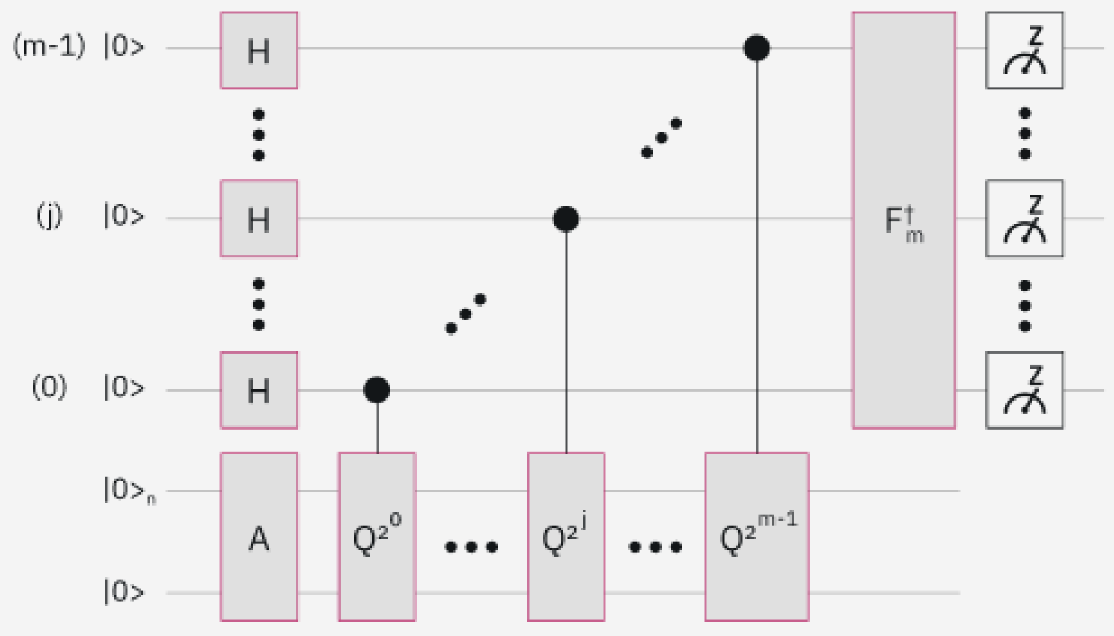
Entanglement forging ansatz | Simulating nature

Mathematics and data with structure



QSVM transpiled job circuit | Mathematics and data with structure

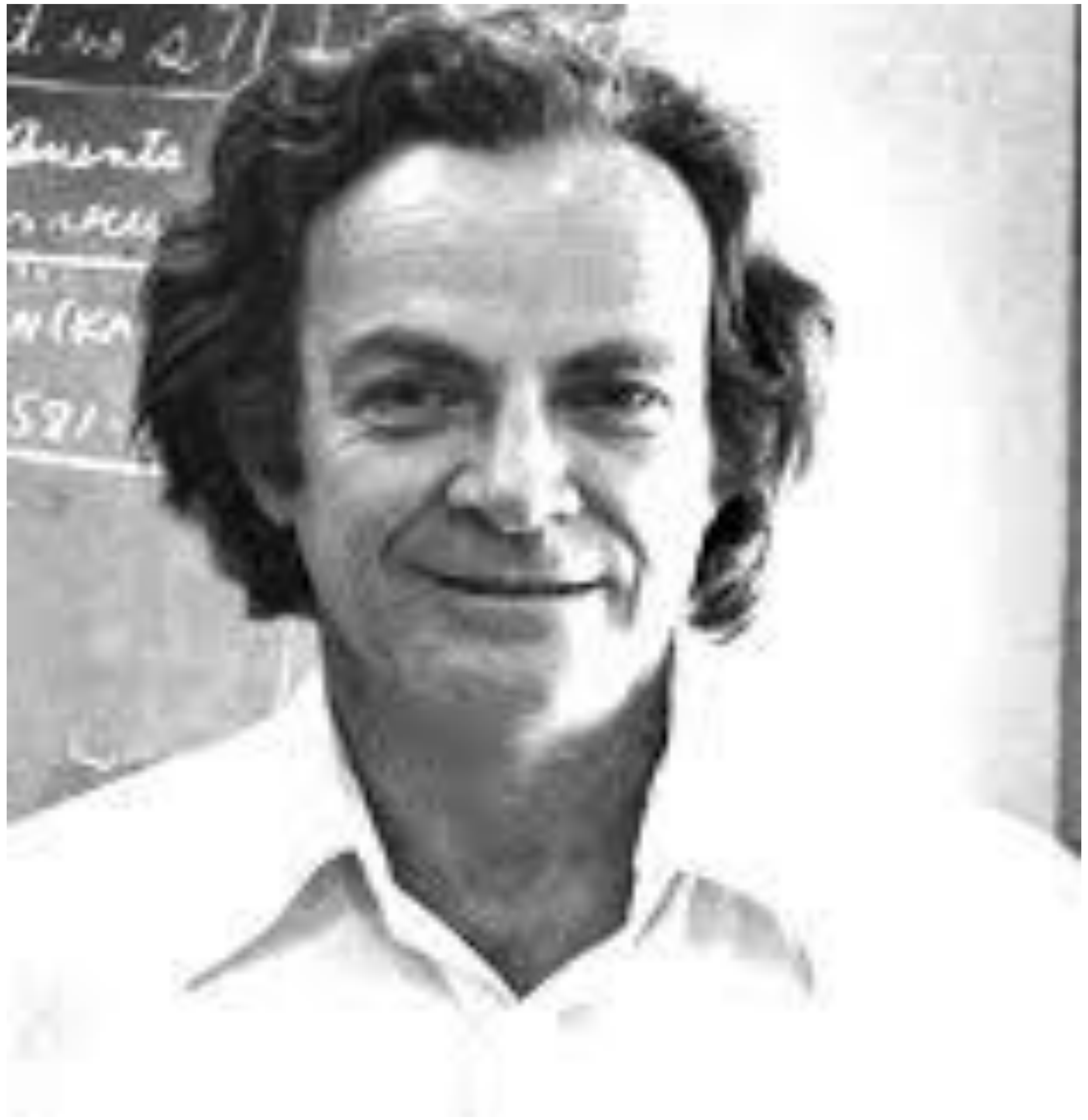
Search and optimization



Amplitude estimation | Search and optimization

Richard Feynman (1981):

“Nature isn't classical, dammit, and if you want to make a simulation of nature, you'd better make it quantum mechanical, and by golly it's a wonderful problem, because it doesn't look so easy.”



Problem areas where quantum computing adds value

Simulating nature

- Spin models
- Materials
- Chemistry
- High energy physics

Mathematics and data with structure

- Factoring
- Abelian hidden subgroup
- Supervised learning (classification)
- Unsupervised learning (clustering)
- Time series
- Regression
- Linear algebra

Search and optimization

- Combinatorial optimization
- Black-box optimization
- General mixed integer

Problem areas where quantum computing adds value

Simulating nature

- Spin models
- Materials
- Chemistry
- High energy physics

Mathematics and data with structure

- Factoring
- Abelian hidden subgroup
- Supervised learning (classification)
- Unsupervised learning (clustering)
- Time series
- Regression
- Linear algebra

Search and optimization

- Combinatorial optimization
- Black-box optimization
- General mixed integer

Problem areas where quantum computing adds value

Simulating nature

- Spin models
- Materials
- Chemistry
- High energy physics

Mathematics and data with structure

- Factoring
- Abelian hidden subgroup
- Supervised learning (classification)
- Unsupervised learning (clustering)
- Time series
- Regression
- Linear algebra

Search and optimization

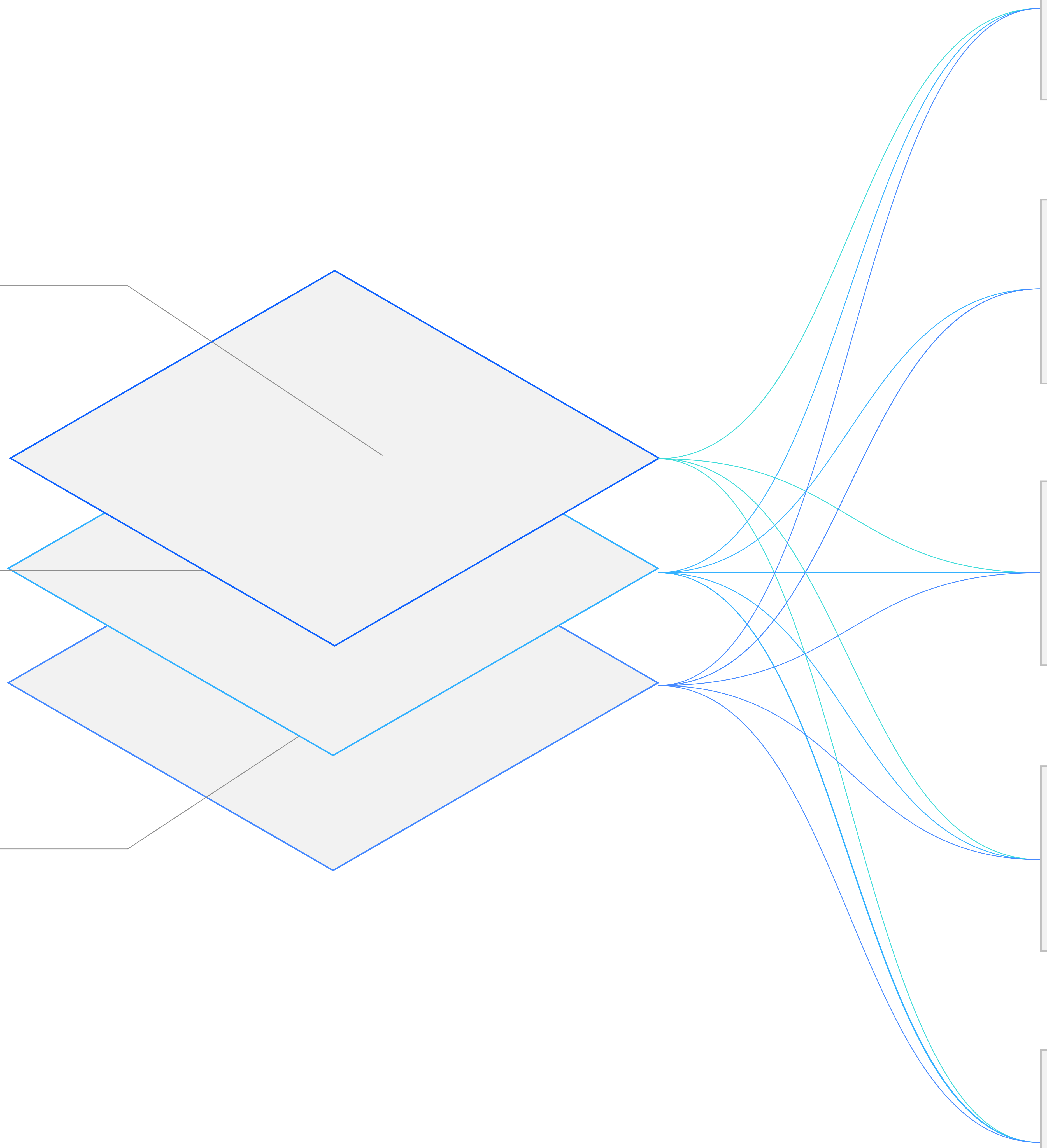
- Combinatorial optimization
- Black-box optimization
- General mixed integer

Quantum computing is expected to have impact across industries

Simulating nature

Mathematics and processing data with complex structure

Search and optimization



Aerospace & Automotive

- Customer Experience
- **Materials Design**
- Structural Design Optimization
- And more

Financial Services

- Fraud Detection
- Derivatives/options pricing
- **Portfolio optimization**
- Risk analysis
- And more

High Tech

- Seismic Imaging
- Catalysts
- Supply chain planning
- Manufacturing scheduling
- And more

Energy, Environment, Utilities

- **Portfolio optimization**
- Grid optimization
- Risk Analysis and Options Pricing
- **Battery Design**
- And more

Health Care & Life Sciences

- Disease risk prediction
- **Drug discovery and design**
- Protein folding predictions
- And more

The quantum utility era

What applications can be tackled with current quantum computers?

The scale of current quantum algorithms can be loosely characterized by:

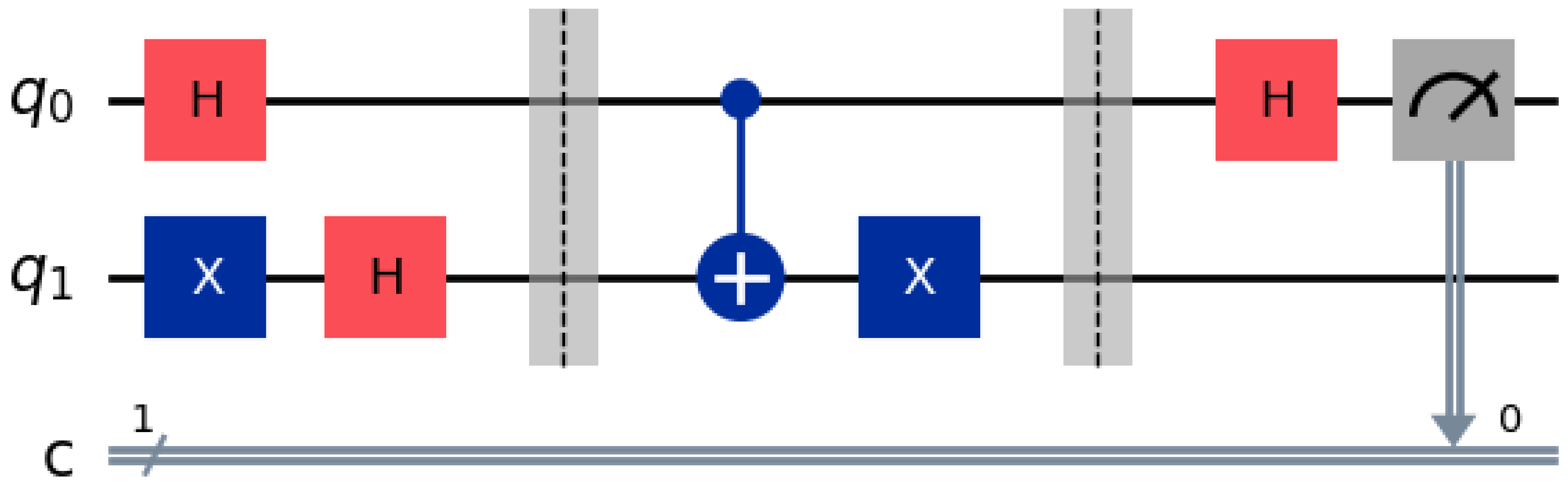
- Number of qubits (i.e. circuit width): determines dimensionality of state space = 2^n
- Number of operations: how many entangling gates can be executed while getting meaningful results? This determines the length of the calculation.

Hardware noise limits how many operations can be carried out

Quantum circuit operations are noisy: for each operation, there is a non-zero probability of introducing an error

Individual errors are small (e.g. $\sim 0.1\%$), but the probability for the calculation to fail is proportional to the number of operations

The lower these errors, the deeper the quantum circuit that can be executed without the result being spoiled by noise

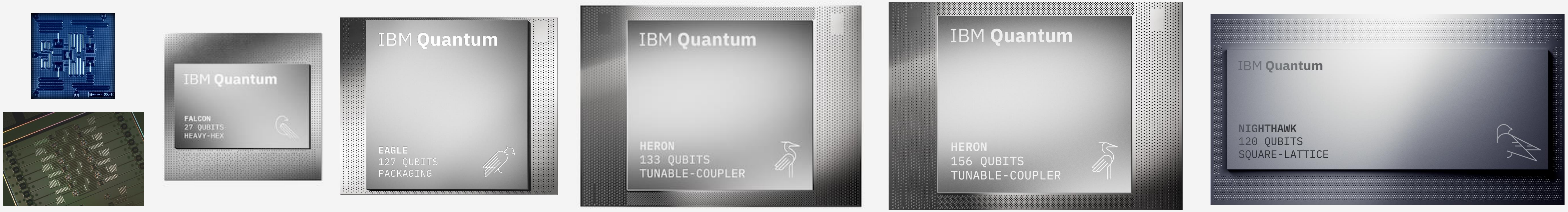


Despite hardware noise, current quantum computers can perform intermediate-scale computations

Number of qubits: > 100 qubits
Number of operations: > 5k entangling gates

Since 2016, IBM has publicly deployed:
- ✓ 60 quantum devices (<100 qubits)
- ✓ 29 quantum computers (>100 qubits) that can run circuits with >5,000 two-qubit gates

Utility-scale



Early | 2019 | 2021 | 2023 | 2024 | 2025

We are now in the era of utility-scale quantum computing

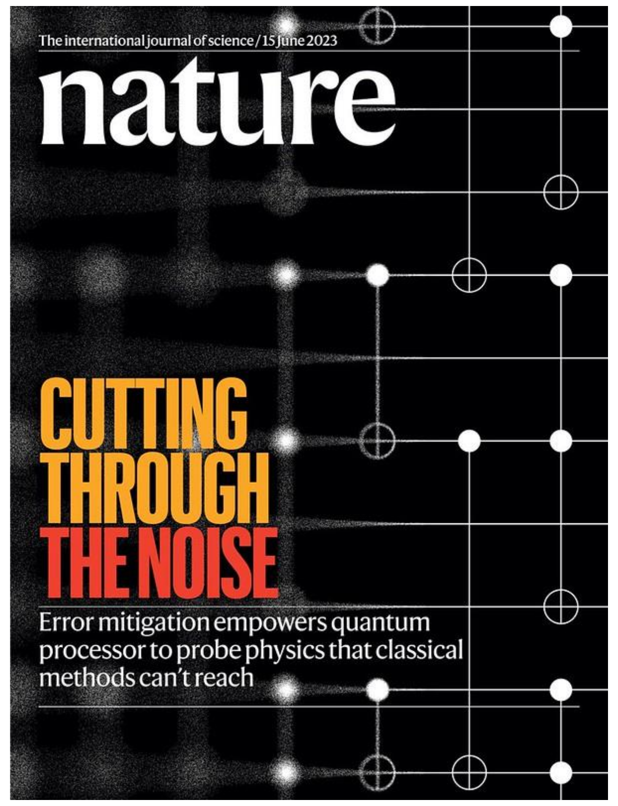
Theory | Utility | Advantage

Quantum Utility (2023)

Demonstration that a quantum computer can run quantum circuits beyond the ability of a classical computer simulating a quantum computer

Quantum Advantage (TBD)

Demonstration that a quantum computer can solve a problem more accurately, cheaper, or more efficiently than classical computing alone

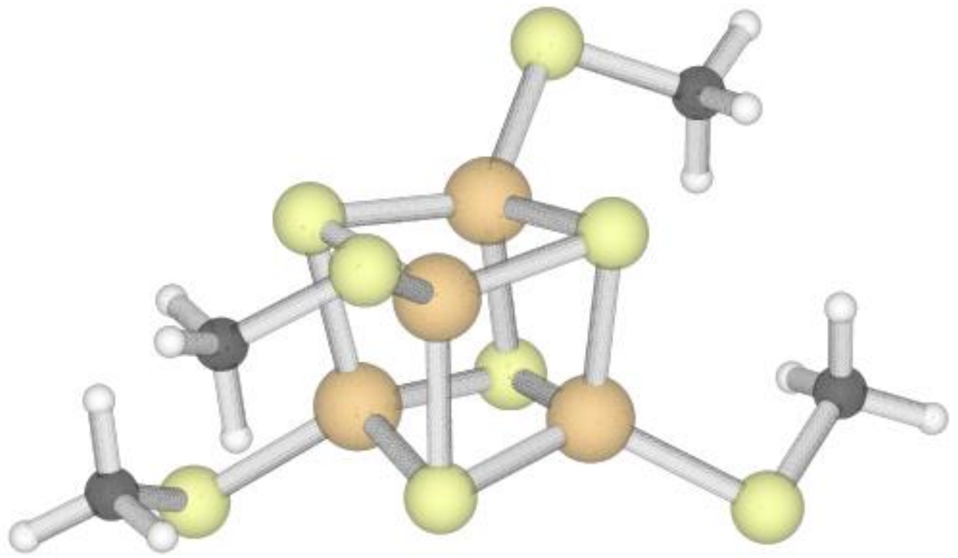


IBM's 2023 research paper ("Evidence for the utility of quantum computing before fault tolerance") provided evidence and methods to move the industry into the Utility era

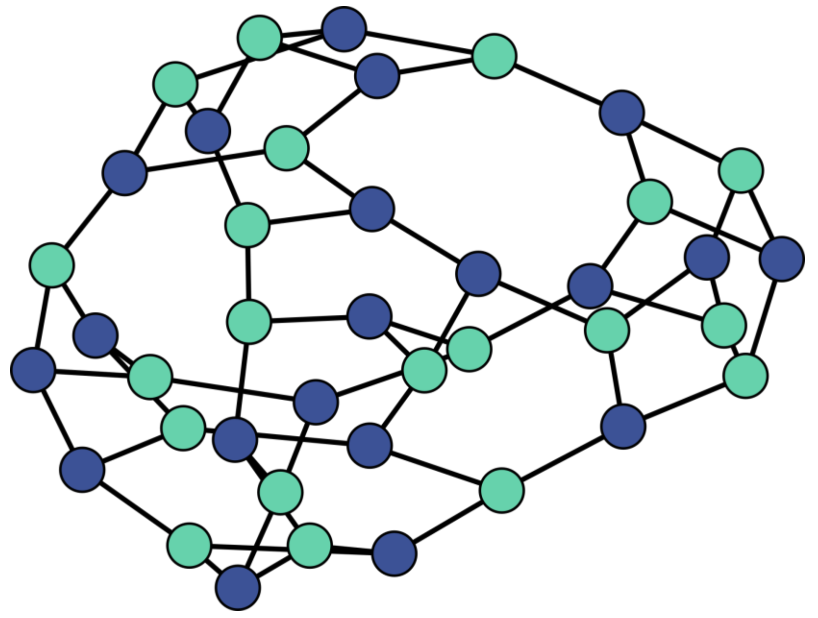
https://www.nature.com/articles/s41586-023-06096-3

Key quantum computational areas (current/near-term)

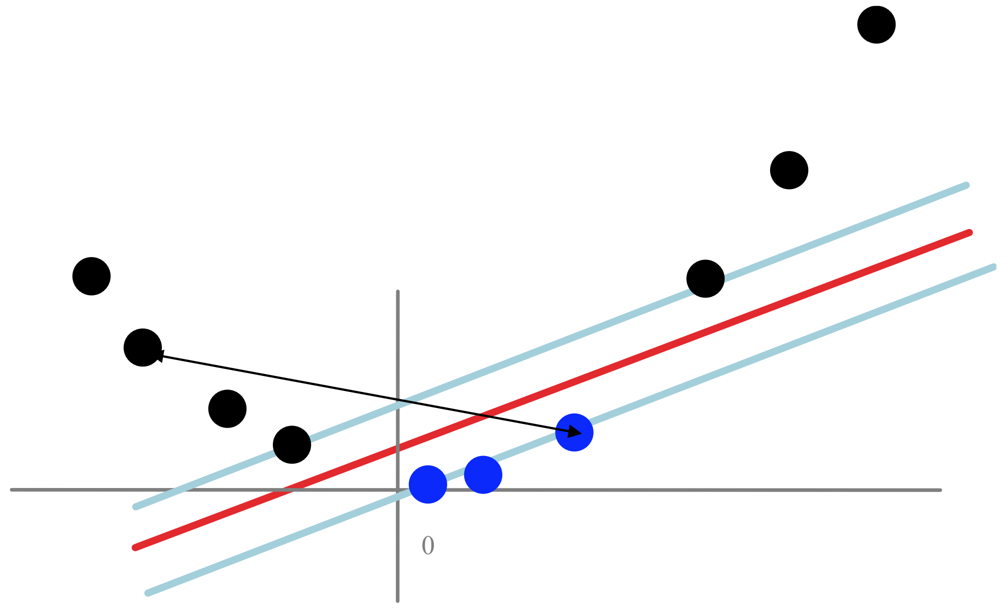
Hamiltonian simulation



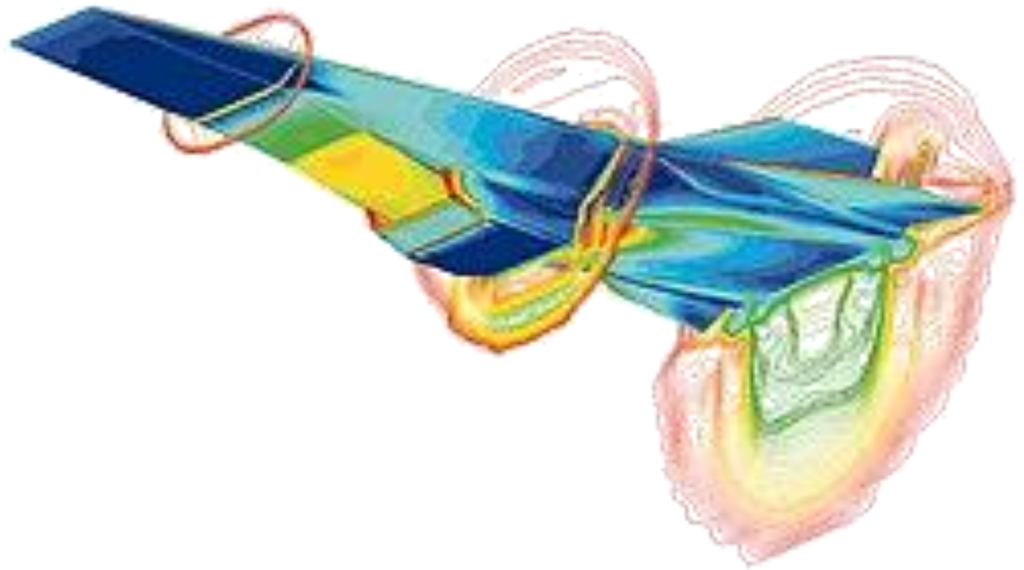
Optimization



Machine learning



Differential equations



How are we going to scale to calculations requiring 100s of millions or billions of operations?

Fault-tolerant quantum computing

Challenge

We need accurate results after 100s millions or more operations

Solution

Error correction: encoding logical qubits in error correcting codes allows one to detect and correct errors *during* circuit execution at the cost of an overhead in the number of qubits

Fault-tolerant quantum computing: implements error correcting codes in such a way that both the error correction and a universal set of logical operations can be carried out in a way that reduces errors

Development Roadmap

2016–2019 ✓	2020 ✓	2021 ✓	2022 ✓	2023 ✓	2024 ✓	2025 ✓	2026	2027	2028	2029	2033+
Ran quantum circuits on IBM Quantum Platform	Released multi-dimensional roadmap publicly with initial focus on scaling	Enhanced quantum execution speed by 100x with Qiskit Runtime	Brought dynamic circuits to unlock more computations	Enhanced quantum execution speed by 5x with Quantum Serverless and execution modes	Demonstrated accurate execution of a quantum circuit at a scale beyond exact classical simulation (5K gates on 156 qubits)	Deliver quantum + HPC tools that will leverage Nighthawk, a new higher-connectivity quantum processor able to execute more complex circuits	Enable the first examples of quantum advantage using a quantum computer with HPC	Improve quantum circuit quality to allow 10K gates	Improve quantum circuit quality to allow 15K gates	Deliver a fault-tolerant quantum computer with the ability to run 100M gates on 200 qubits	Beyond 2033, quantum computers will run circuits comprising a billion gates on up to 2000 qubits, unlocking the full power of quantum computing

Code assistant ✓ (2024–2033+)

Applying algorithms to applications
- Functions ✓ (2024–2025)
- Use case benchmarking toolkit (on target) (2026)
- Computation libraries (2027–2033+)

Discovering new algorithms for advantage
- Advanced classical transpilation tools ✓ (2024)
- Advanced classical mitigation tools ✓ (2025)
- Utility mapping tools (on target) (2026–2028)
- Circuit libraries (2029–2033+)

Orchestrating workloads for quantum + HPC
- Resource Management (2023–2033+)
- Qiskit Serverless ✓ (2023)
- Plugins for HPC ✓ (2024)
- C API ✓ (2025)
- Execution modes ✓ (2023–2025)
- Profiling tools (on target) (2026–2027)
- Workflow accelerators (2028–2033+)

Accurately and efficiently executing on quantum computers
- Qiskit Runtime (2021–2033+)
- IBM Quantum Experience ✓ (2016–2020)
- OpenQASM 3 ✓ (2021)
- Dynamic Circuits ✓ (2022)
- Error mitigation ✓ (2023)
- 200K CLOPS ✓ (2024)
- Utility-scale dynamic circuits ✓ (2025–2028)
- Fault-tolerant ISA (2029–2033+)

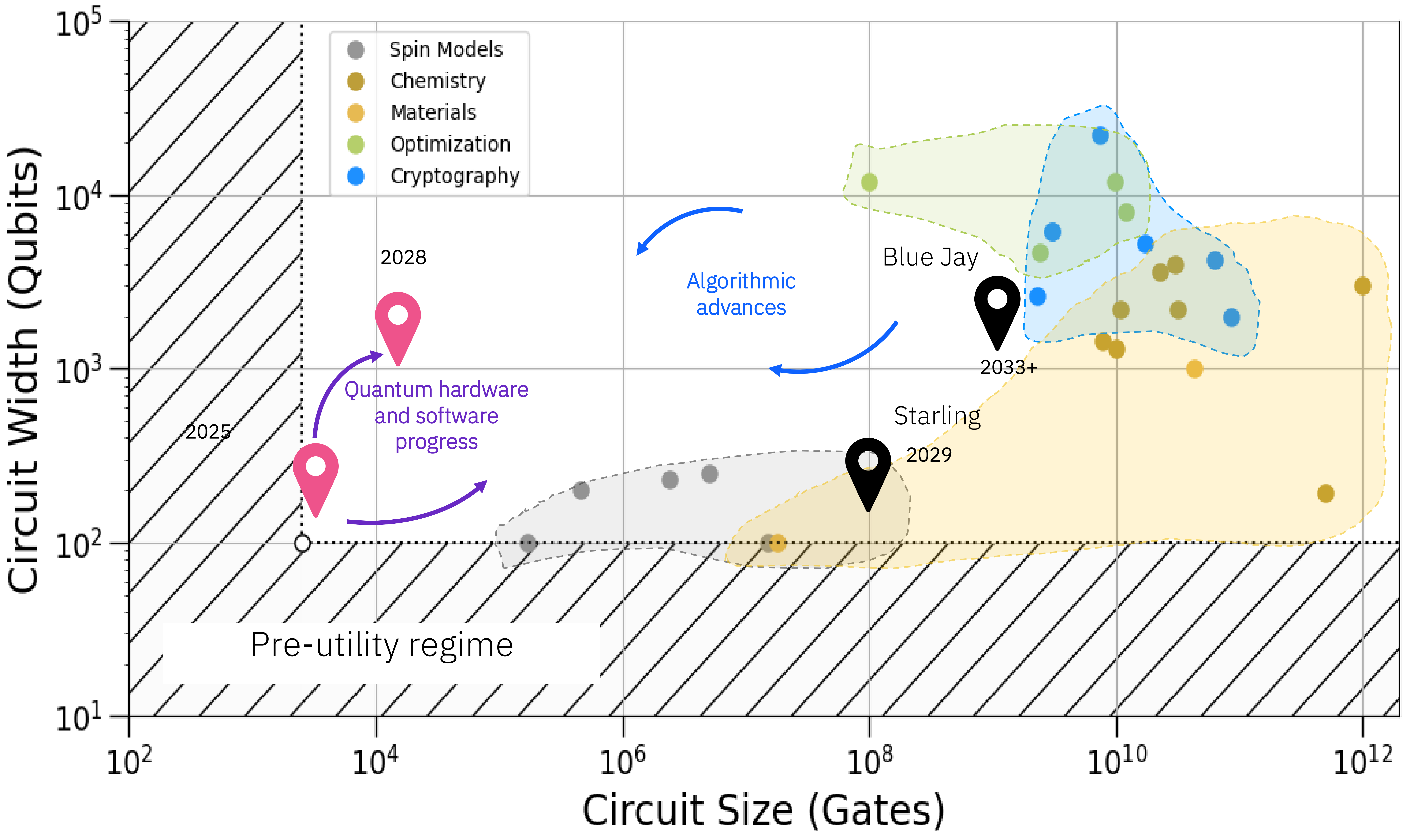
Hardware →

Early ✓	Falcon ✓	Eagle ✓	Heron (5K) ✓	Nighthawk (5K) ✓	Nighthawk (7.5K) (on target)	Nighthawk (10K)	Nighthawk (15K)	Starling (100M)	Blue Jay (1B)
Canary 5 qubits; Albatross 16 qubits; Penguin 20 qubits; Prototype 53 qubits	Benchmarking; 27 qubits	Benchmarking; 127 qubits	Error mitigation; 5K gates, 133 qubits	Error mitigation; 5K gates, 120 qubits	Error mitigation; 7.5K gates, 120 qubits; Up to 120x3 = 360 qubits	Error mitigation; 10K gates, 120 qubits; Up to 120x9 = 1080 qubits	Error mitigation; 15K gates, 120 qubits; Up to 120x9 = 1080 qubits	Fault-tolerant; 100M gates, 200 qubits	Fault-tolerant; 1B gates, 2000 qubits

✓ Completed
(on target) On target

Completed
On target

What can you do with a large-scale, fault-tolerant quantum computer?



Thank you